

An Introduction to Noisy Intermediate Scale Quantum (NISQ) Quantum Machine Learning (QML)

We Will Cover

What is Machine Learning

What is Quantum Machine Learning

Fault Tolerant vs NISQ Era

Data Encoding

Classical and Quantum Support Vector Machines

Classical and Quantum Neural Networks

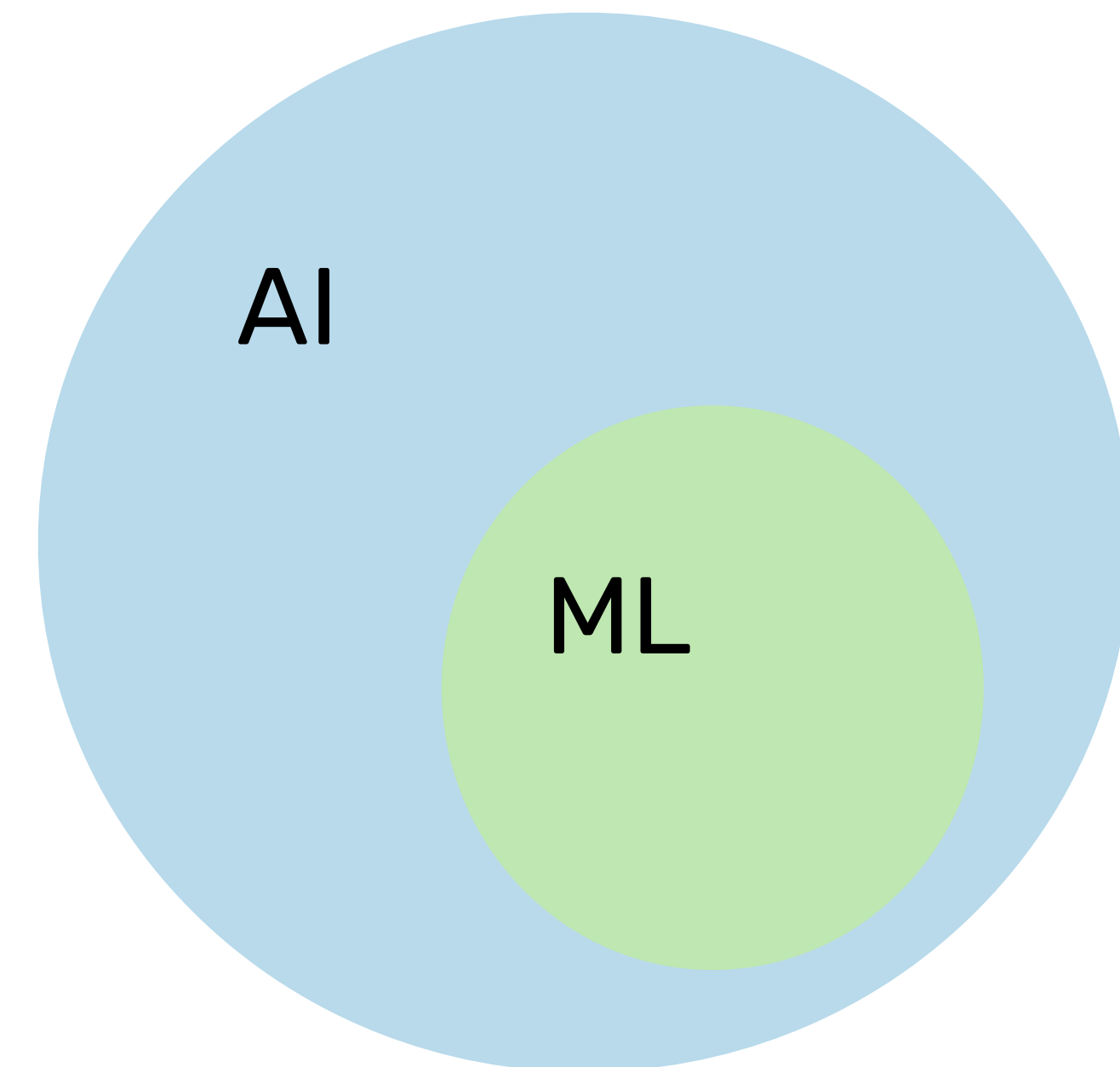
Classical and Quantum Convolutional Neural Networks

What is Machine Learning?

Machine Learning is a method in Artificial Intelligence

Artificial Intelligence: making machines and computer systems perform human-like tasks such as reasoning, navigating and interacting

Machine Learning: one particular method in AI, focused on designing computer algorithms that generalise from patterns found in data to make predictions in unknown situations



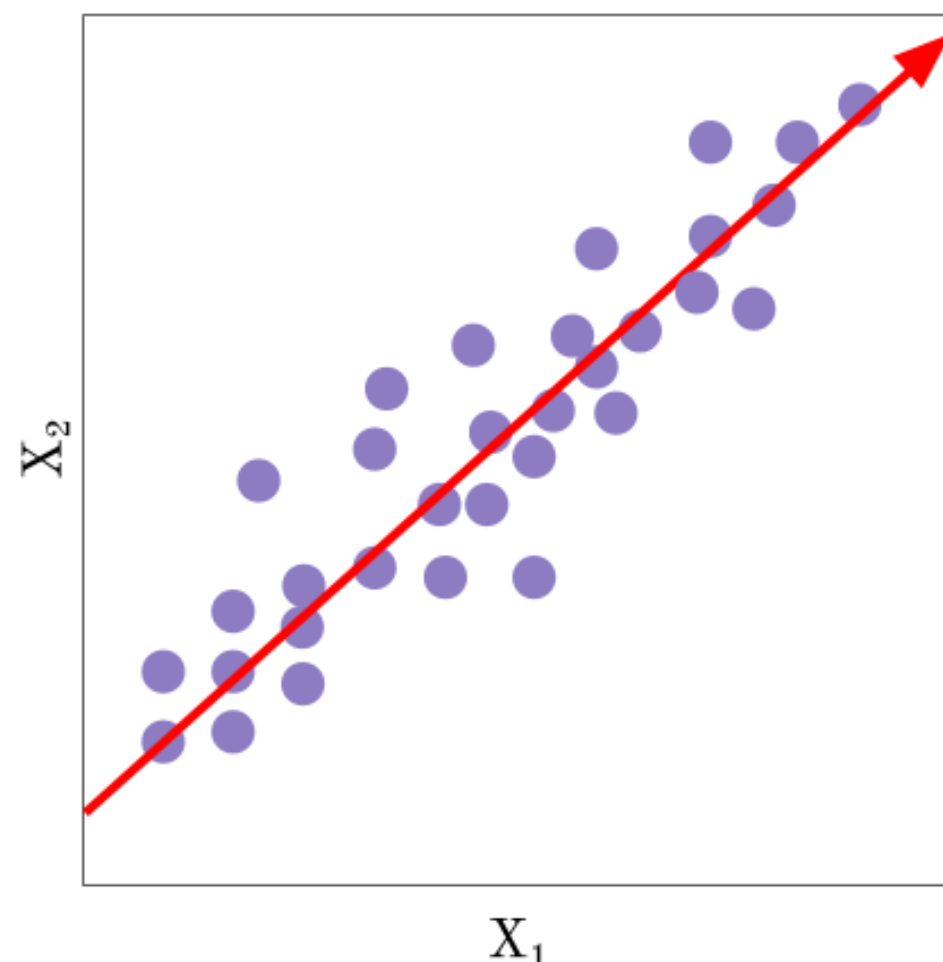
Important Developments in Machine Learning

1913	Early N-Grams Foundations (Markov, 1913)
1943	Artificial Neuron (McCulloch & Pitts, 1943)
1948	Formal N-Gram Language Modeling (Shannon, 1948)
1963	SVM Precursor (Vapnik & Lerner, 1963)
1964	Automatic Differentiation (Wengert, 1964)
1966	ELIZA, The First Widely Recognised Chatbot (Weizenbaum, 1966)
1970/1976	Reverse-Mode Automatic Differentiation (Linnainmaa, 1976)
1974/1986	Backpropagation (Werbos, 1974; Rumelhart et al., 1986)
1989	Universal Approximation Theorem (Cybenko, 1989)
1980	Early CNN Architecture (Fukushima, 1980)
1989/1998	Modern CNNs (LeCun et al., 1989, 1998)
1992/1995	Modern SVMs (Cortes & Vapnik, 1995)
2017	Transformer Architecture (Vaswani et al., 2017)

3 Types of Machine Learning

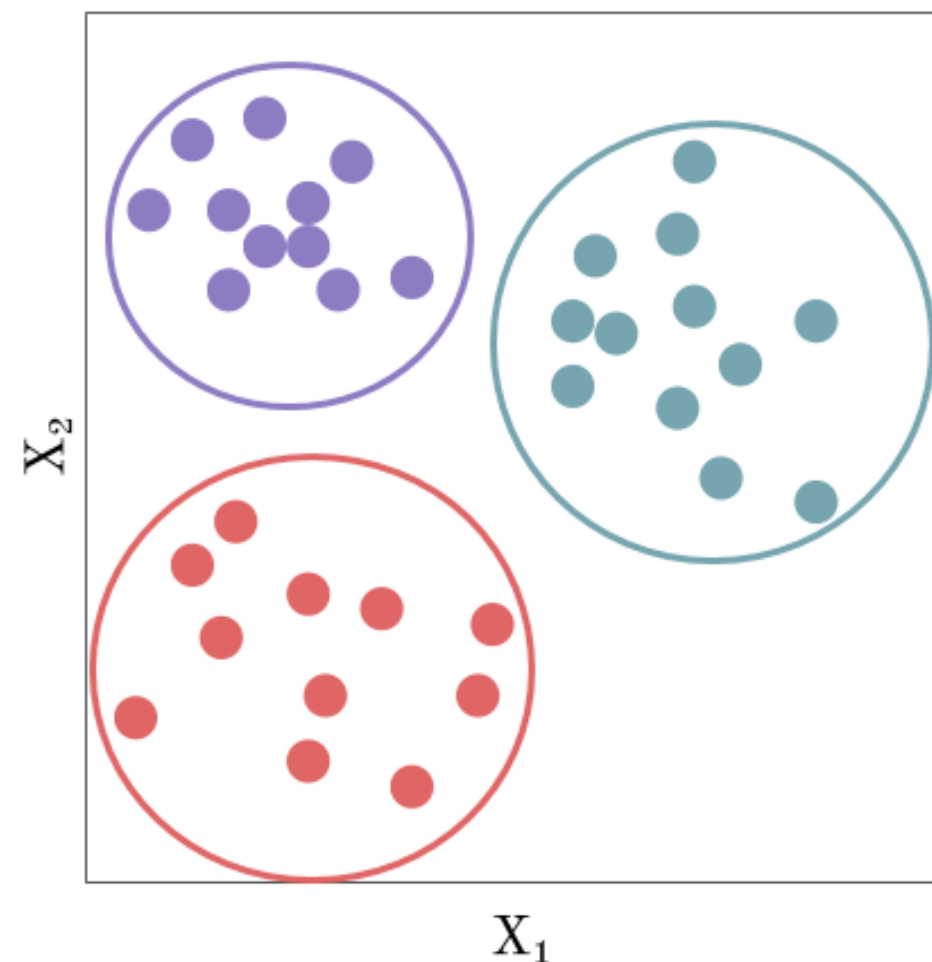
Supervised Learning

Data consists of input-output pairs. Goal is to predict output of unseen input.



Unsupervised Learning

Data does not come with target outputs. Goal is to find patterns or structure in the data.

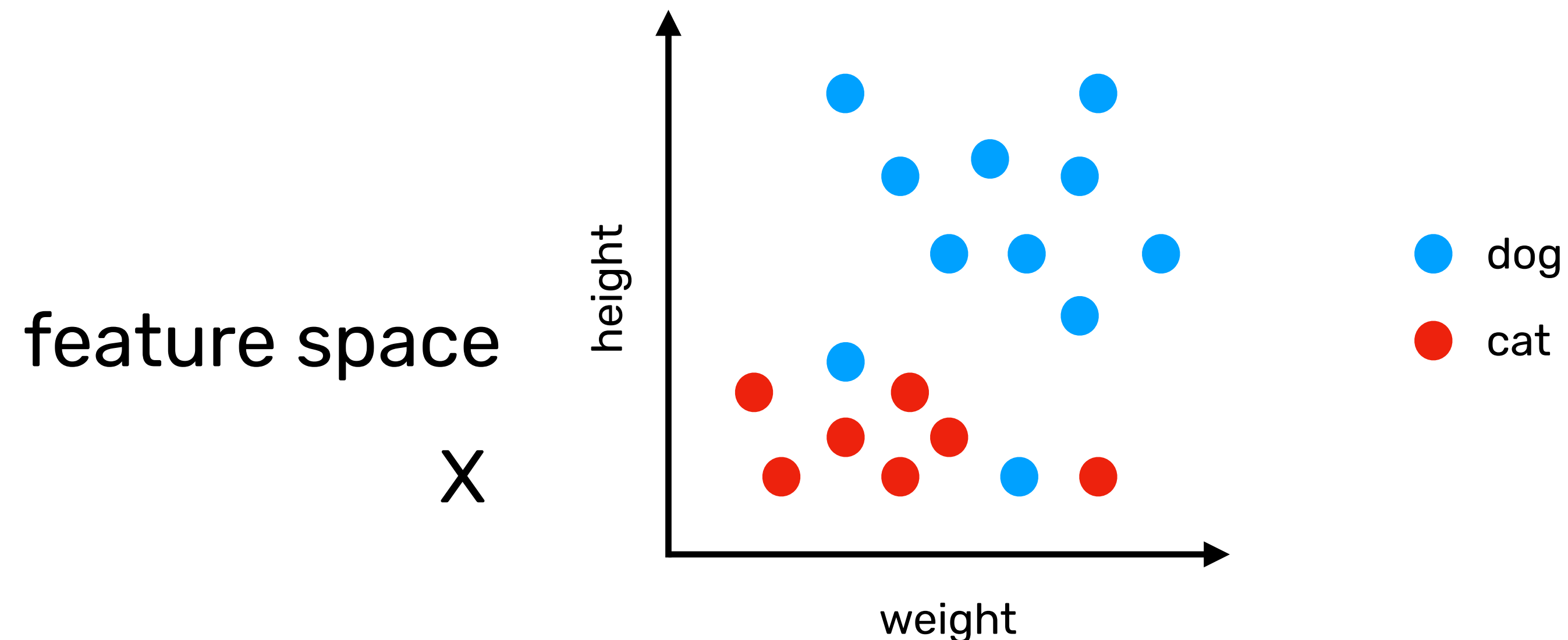


Reinforcement Learning

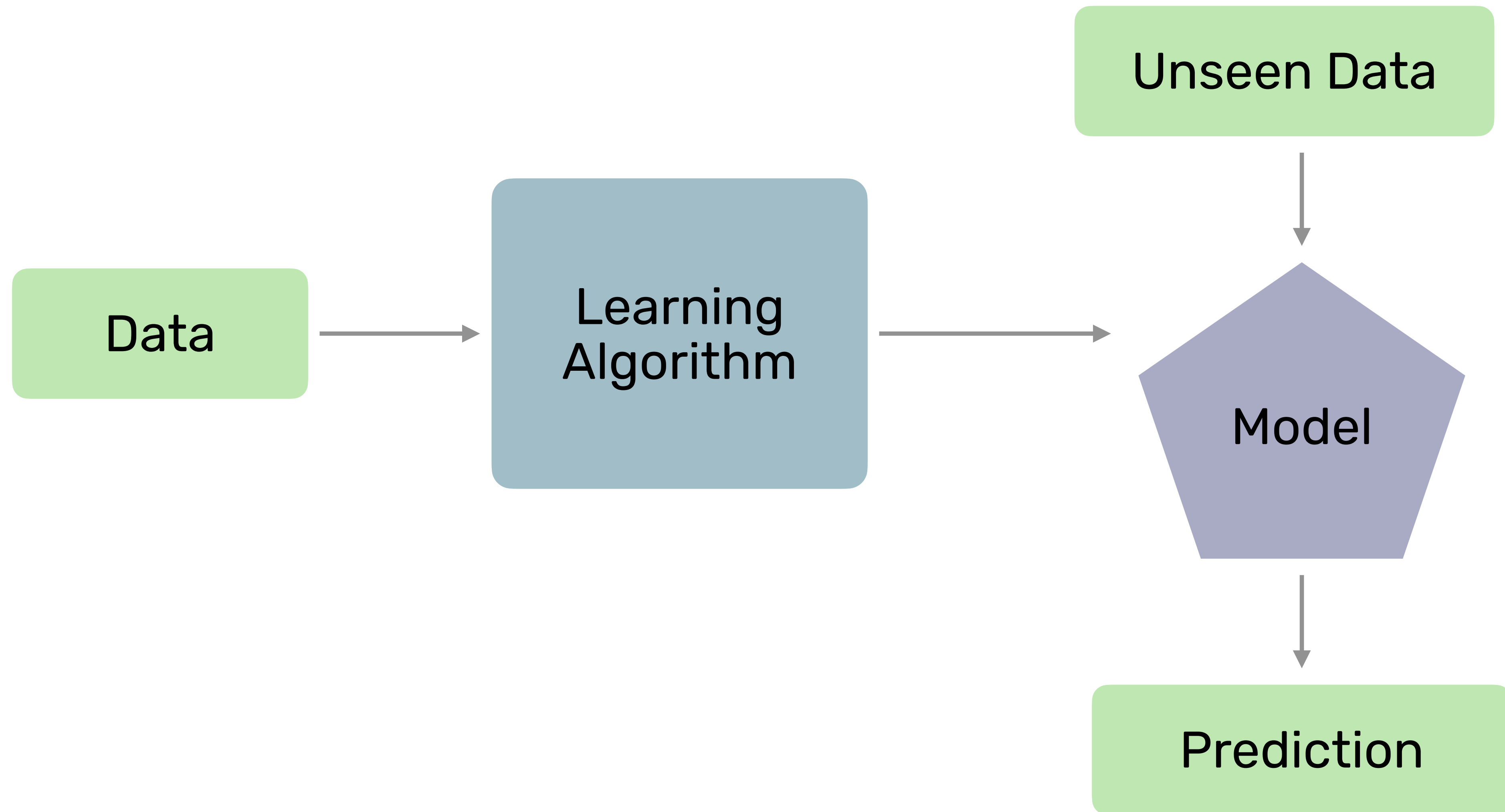
An *agent* navigates an *environment* and gets rewarded or penalised according to a given rule for its decisions. The agent learns an optimal strategy by trial and error.

The Supervised Learning Task

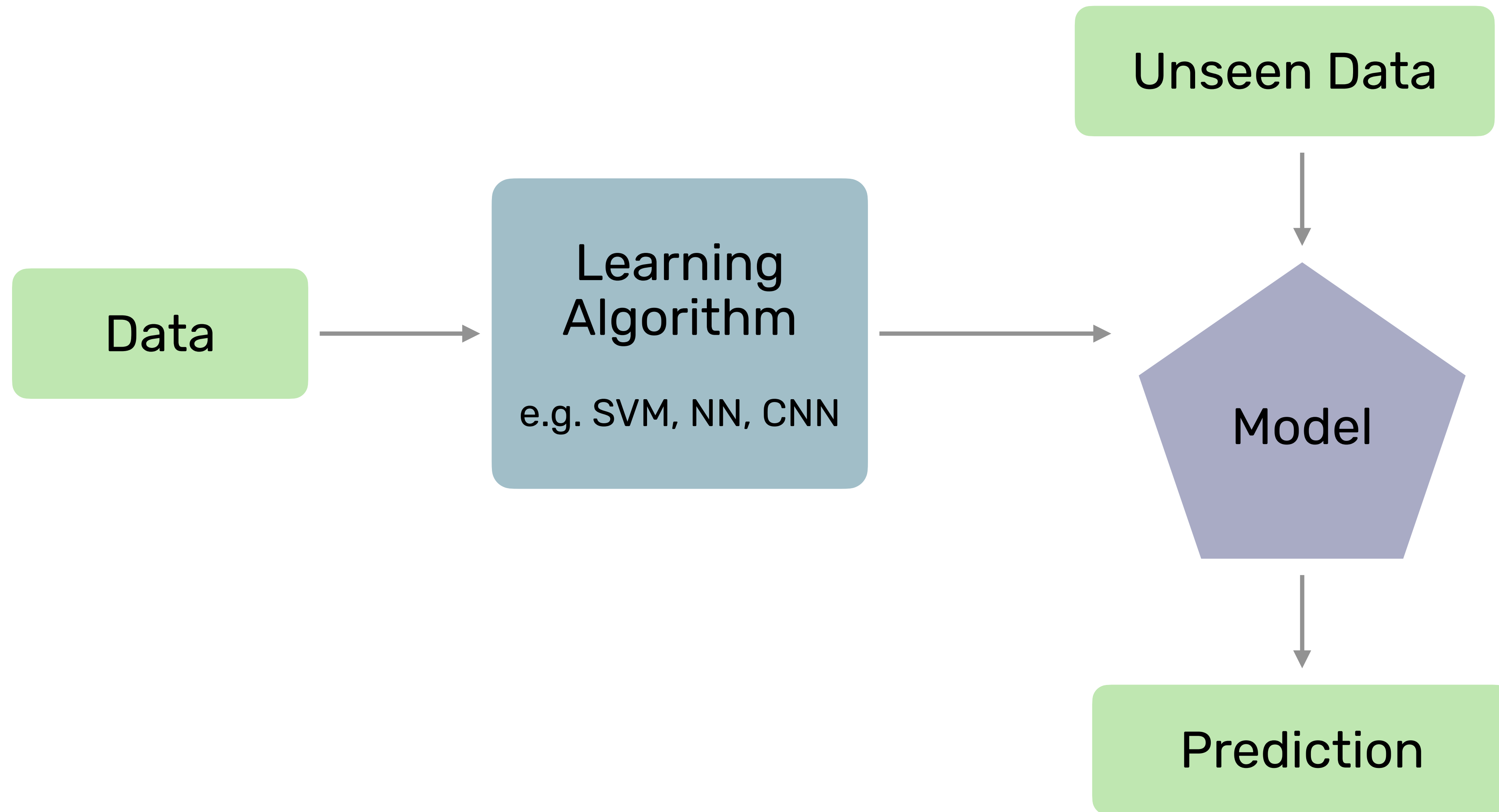
Supervised Learning Task: Given an input domain $\mathcal{X}$, an output domain $\mathcal{Y}$, a data set $\mathcal{D} = \{(\mathbf{x}^1, y^1), \dots, (\mathbf{x}^M, y^M)\}$ of pairs $(\mathbf{x}^m, y^m) \in \mathcal{X} \times \mathcal{Y}$ that contain inputs $\mathbf{x}^m$ and target outputs y^m as well as a new unclassified input $\tilde{\mathbf{x}} \in \mathcal{X}$, predict the corresponding output $\tilde{y} \in \mathcal{Y}$.



How to Solve Supervised Learning Tasks

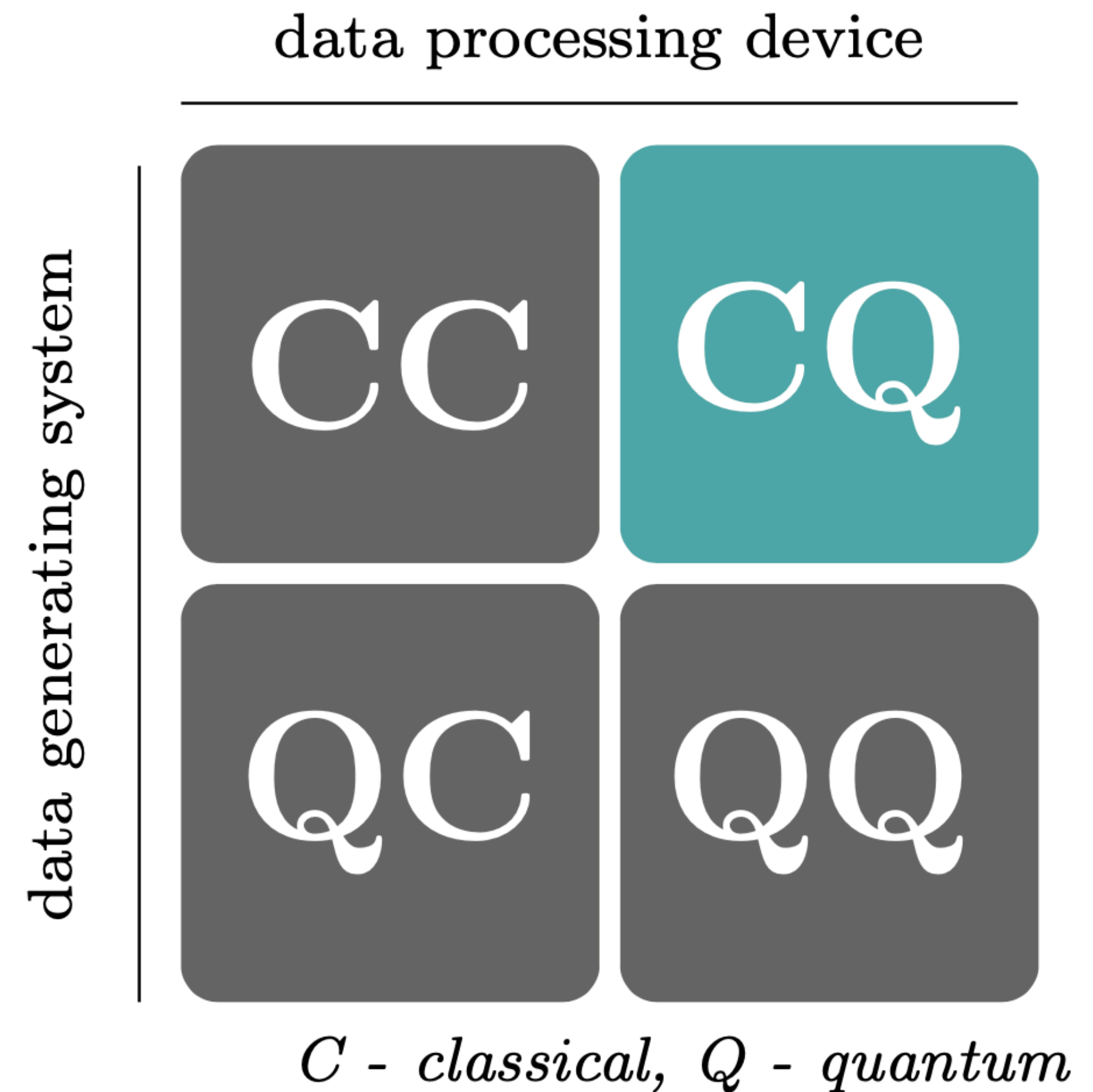


How to Solve Supervised Learning Tasks



What is Quantum Machine Learning?

Quantum Machine Learning seeks to combine techniques or theories from quantum mechanics and machine learning



Fault-Tolerant Setting vs NISQ Era

Now

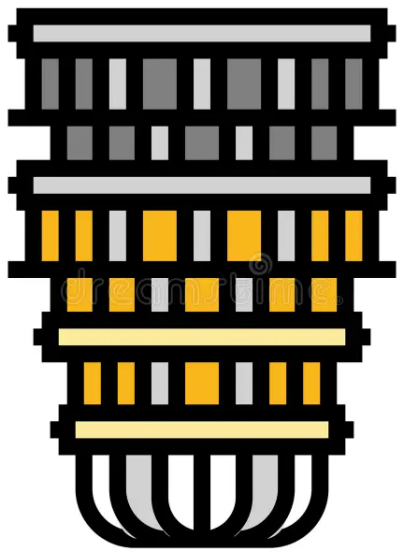
The Future

NISQ (Noisy Intermediate Scale Quantum) Era: refers to a transitional period in the development of quantum computers, characterised by the availability of intermediate-scale quantum devices that are not yet fault-tolerant. The quantum devices and the operations performed on them are prone to noise.

Fault-Tolerant Setting: refers to a state in which quantum computers are designed and operated to minimise the impact of errors and noise that naturally occur in quantum systems

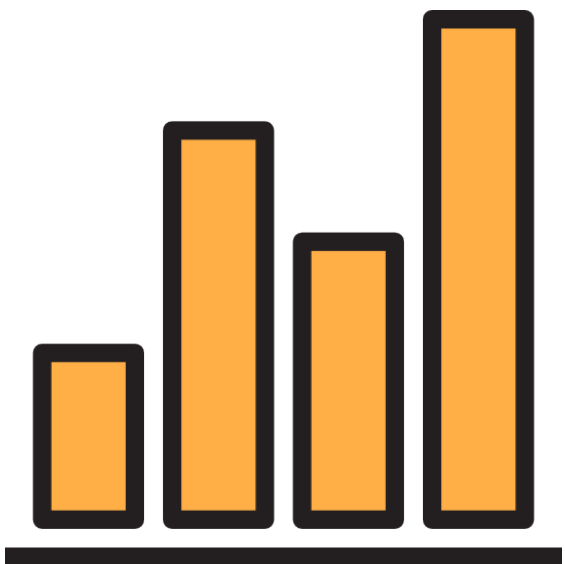
Fault-Tolerant Setting vs NISQ Era

There are 3 sources of error in the NISQ era:



Device Noise: results from the inevitable interaction of the quantum system with its environment as well as imperfect gate implementation

Readout Error: error due to the measurement of an unexpected state



Finite Sampling: the statistical uncertainty caused by estimating measurement probabilities from a limited number of circuit executions (“shots”)

Fault-Tolerant Setting vs NISQ Era

There are 3 sources of error in the NISQ era.

Solution (FOR NOW):



Post-processing techniques



Run quantum algorithms that require a few number of qubits (short width) and a few number of gates/operations (short depth)

Motivations in Quantum Machine Learning

Theme: QML Algorithms for the Fault Tolerant Setting

Goal: find quantum algorithms that can potentially speed up quantum algorithms

Theme: QML Algorithms for the NISQ Era

Goal: Make use of a quantum device as a machine learning model

Theme: Theoretical study of QML algorithms

Goal: develop our understanding of QML algorithms

Motivations in Quantum Machine Learning

Theme: QML Algorithms for the Fault Tolerant Setting

Goal: find quantum algorithms that can potentially speed up quantum algorithms

Theme: QML Algorithms for the NISQ Era

Goal: Make use of a quantum device as a machine learning model

Theme: Theoretical study of QML algorithms

Goal: develop our understanding of QML algorithms

Motivations in Quantum Machine Learning

Theme: QML Algorithms for the Fault Tolerant Setting

Goal: find quantum algorithms that can potentially speed up quantum algorithms

Theme: QML Algorithms for the NISQ Era

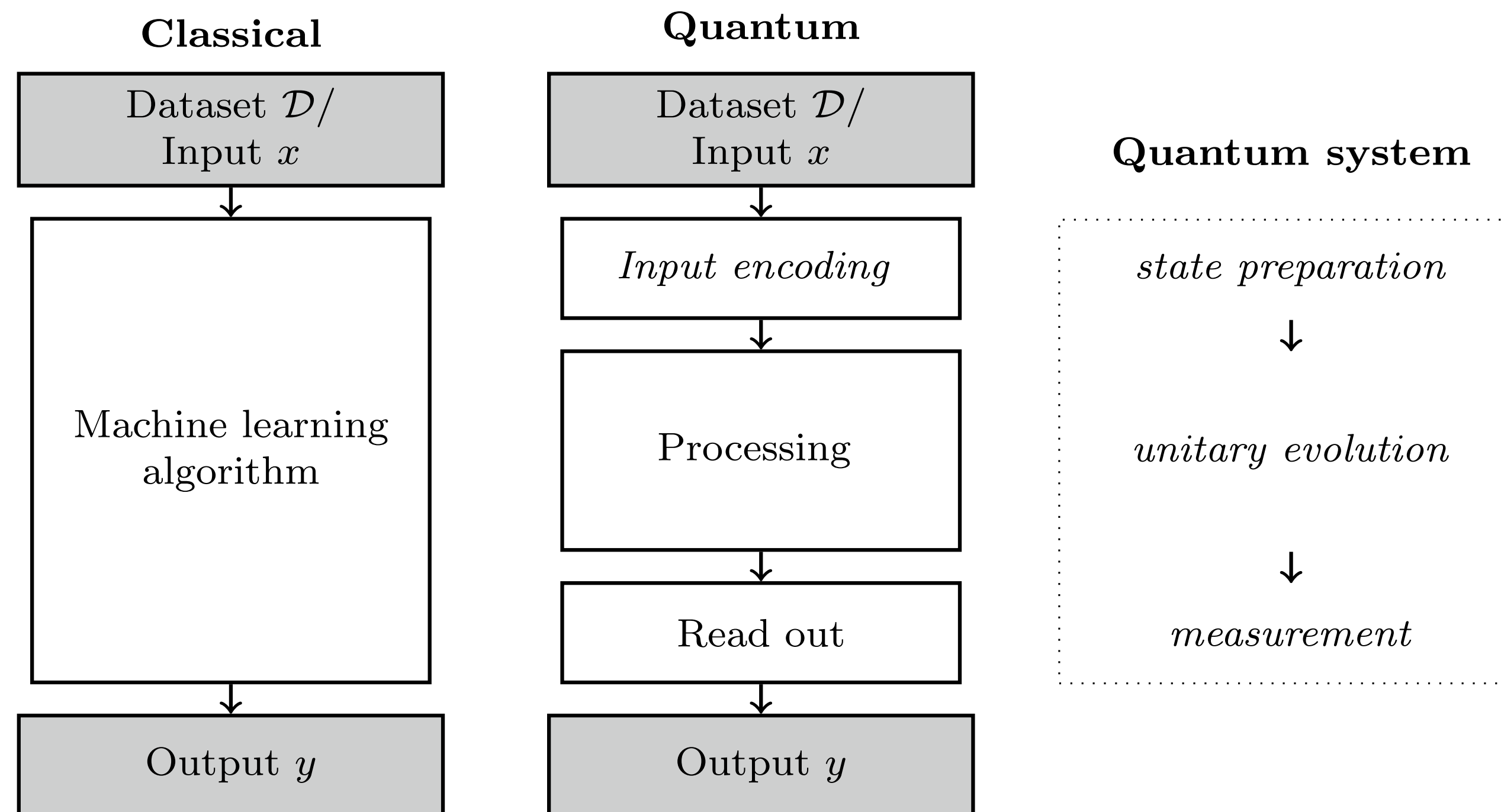
Goal: Make use of a quantum device as a machine learning model

Theme: Theoretical study of QML algorithms

Goal: develop our understanding of QML algorithms

Data Encoding

If we want to process classical data on a quantum computer, we first have to decide how to represent data by quantum states.



Data Encoding

There are many different ways to encode or embed information into a quantum state. Let's look at some examples:

Basis Encoding

Classical Data

$$\mathbf{x} \in \{0,1\}^{\otimes n}$$

Example:

$$\mathbf{x} = \{0,1,1\}$$

Quantum State

$$|\psi\rangle = |x_1, \dots, x_n\rangle$$

$$|\psi\rangle = |011\rangle$$

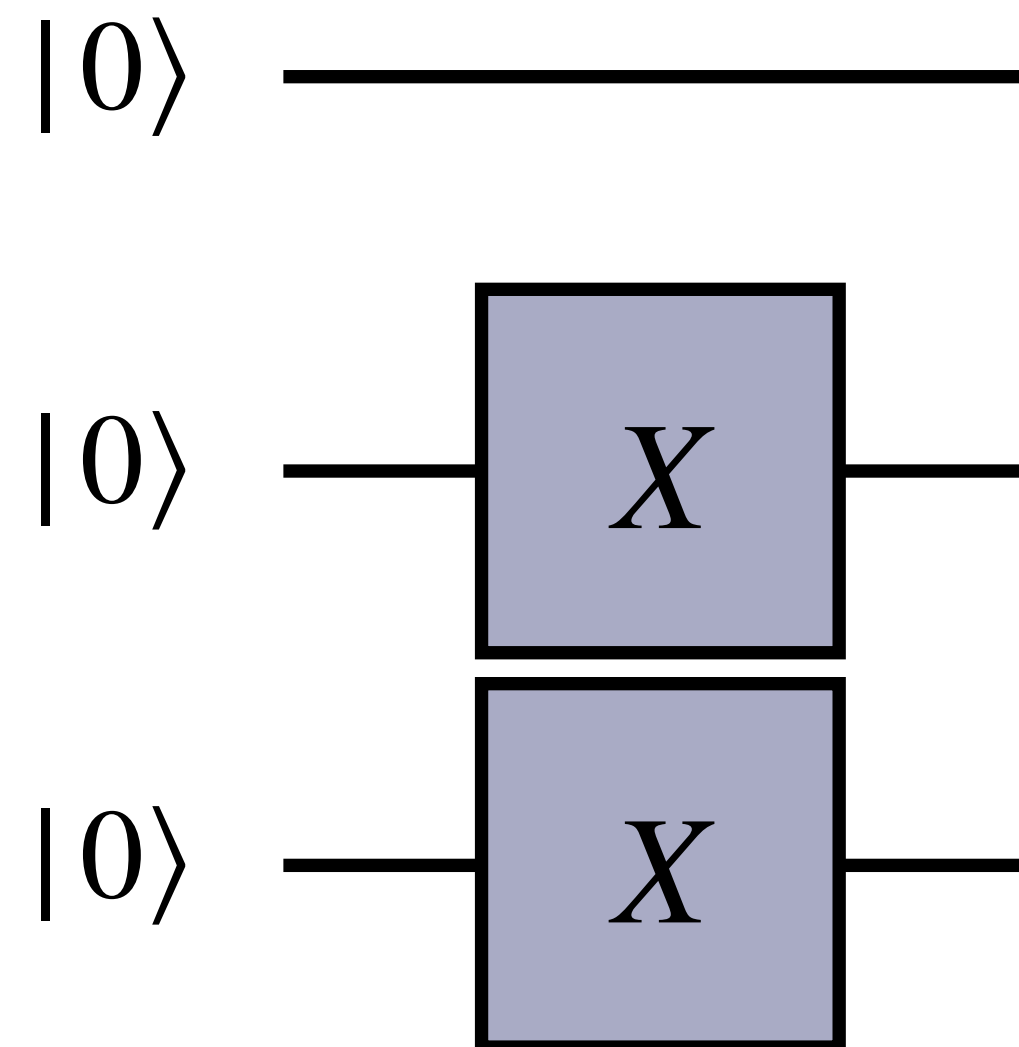
Data Encoding

There are many different ways to encode or embed information into a quantum state. Let's look at some examples:

Basis Encoding

Example:

$$|\psi\rangle = |011\rangle$$



Data Encoding

There are many different ways to encode or embed information into a quantum state. Let's look at some examples:

Amplitude Encoding

Classical Data

$$\mathbf{x} \in \mathbb{R}^n, \sum_k |x_k|^2 = 1$$

Example:

$$\mathbf{x} = \{\sqrt{2}, \sqrt{8}\}$$

Quantum State

$$|\psi\rangle = \sum_{i=0}^{n-1} x_i |i\rangle$$

$$|\psi\rangle = \sqrt{2} |0\rangle + \sqrt{8} |1\rangle$$

Data Encoding

There are many different ways to encode or embed information into a quantum state. Let's look at some examples:

Rotation Encoding

Classical Data

$$\mathbf{x} \in \mathbb{R}^n, \mathbf{x} \in [0, 2\pi]^{\otimes n}$$

Example:

$$\mathbf{x} = \{1, 3\}$$

Quantum State

$$|\psi\rangle = R_y(x_1)|0\rangle \otimes \dots \otimes R_y(x_n)|0\rangle$$

$$|\psi\rangle = R_y(1)|0\rangle \otimes R_y(3)|0\rangle$$

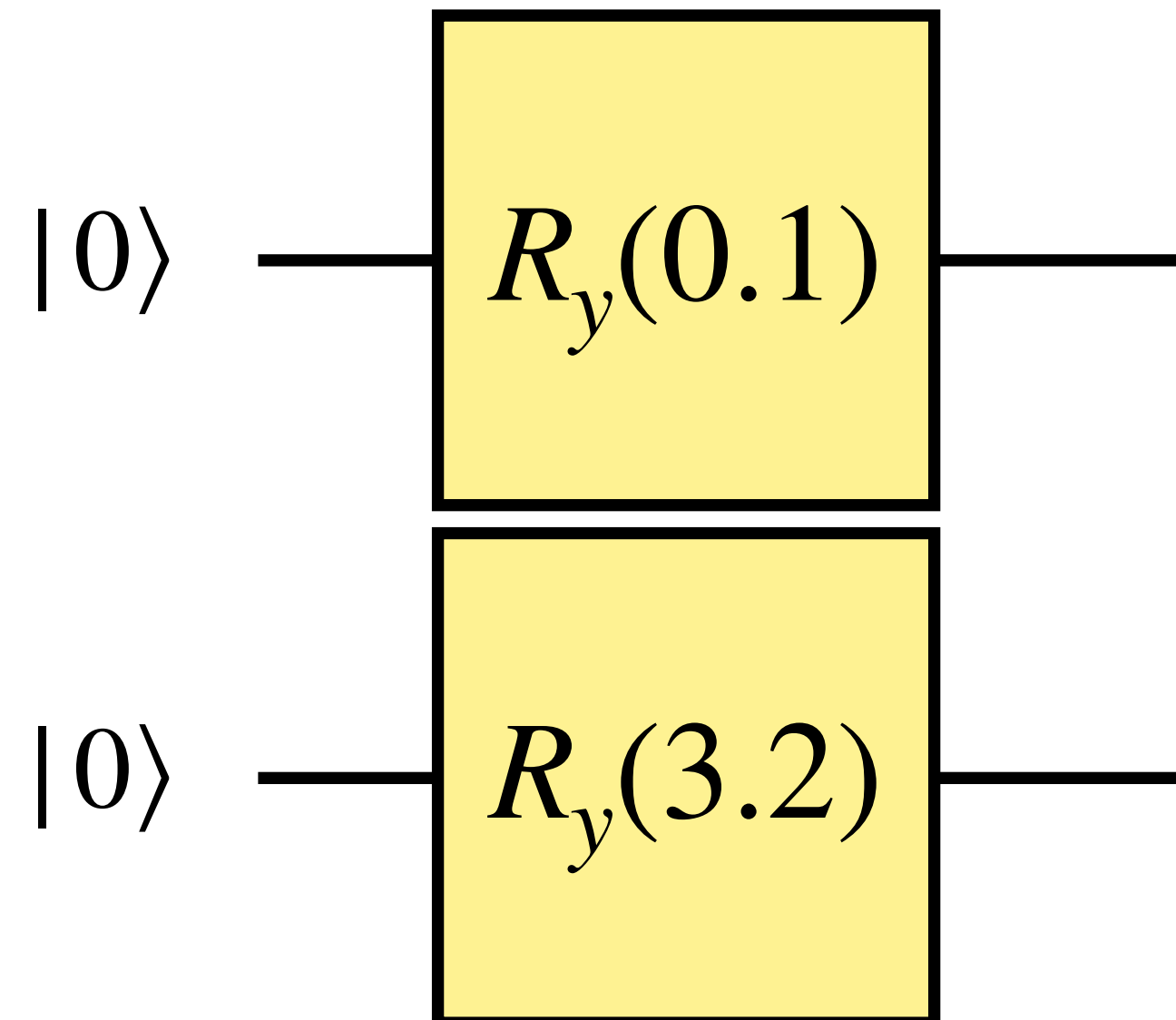
Data Encoding

There are many different ways to encode or embed information into a quantum state. Let's look at some examples:

Rotation Encoding

Example:

$$|\psi\rangle = R_y(0.1)|0\rangle \otimes R_y(3.2)|0\rangle$$



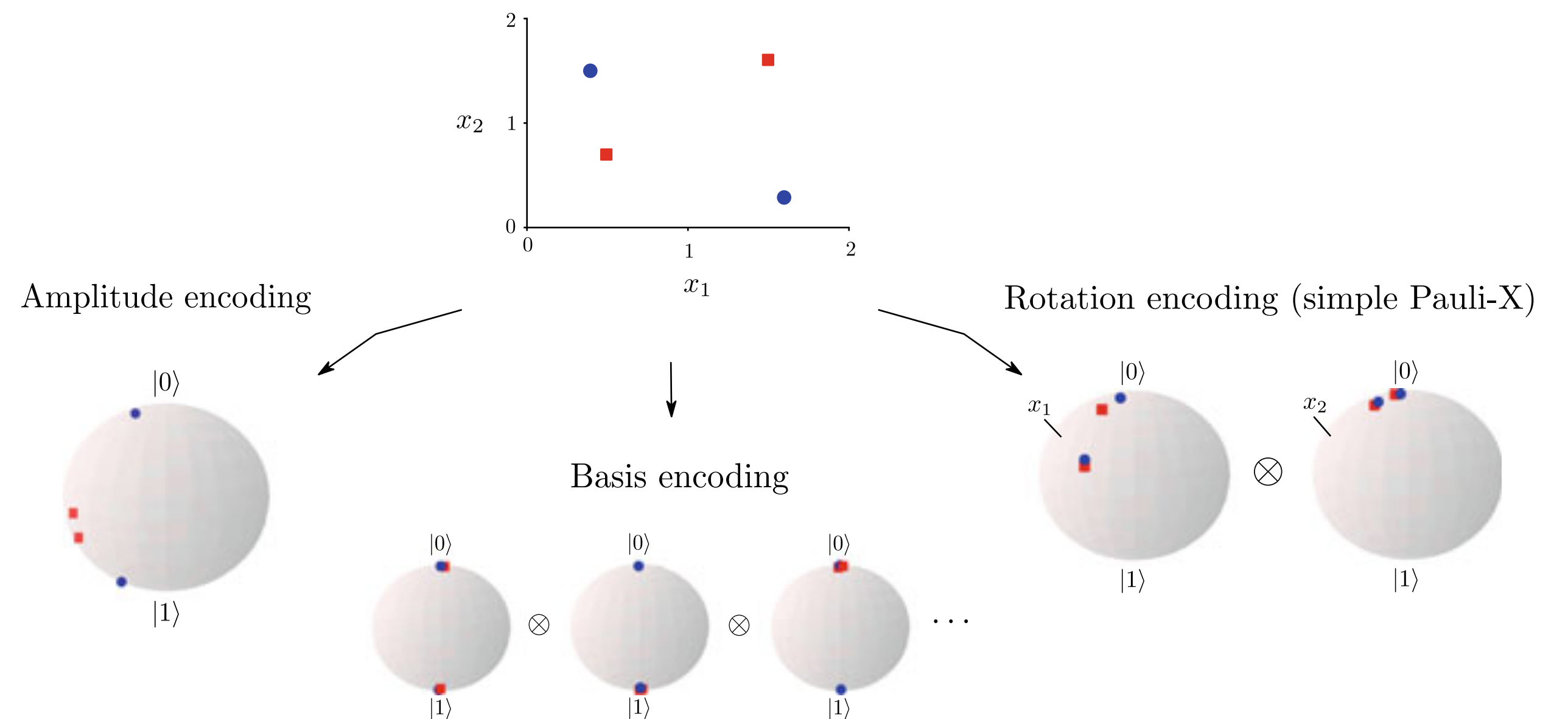
Data Encoding

Encoding a classical data vector into a quantum state

$$\mathbf{x} \rightarrow |\phi(\mathbf{x})\rangle = U_{\phi}(\mathbf{x}) |0\rangle^{\otimes n}$$

can be interpreted as applying a feature map to that vector:

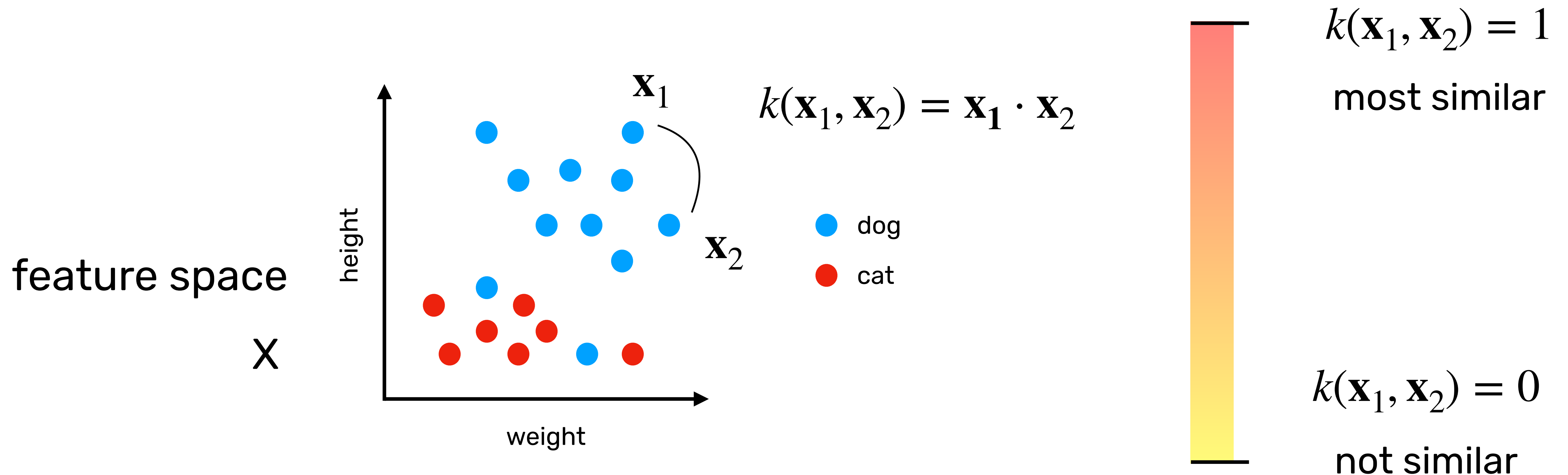
$$\phi : \mathbf{x} \rightarrow |\phi(\mathbf{x})\rangle$$



Classical Support Vector Machine

What is a kernel?

A kernel is a similarity measure defined over the input space.



Classical Support Vector Machine

What is a kernel? (More Formally)

A kernel is a similarity measure defined over the input space.

Given an input space X , a kernel is a positive semi-definite bivariate function:

$$k : X \times X \rightarrow \mathbb{R}$$

Positive semi-definiteness means that for any set $D = \{\mathbf{x}_1, \dots, \mathbf{x}_M\} \subset X$, the Gram or Kernel matrix with entries

$$K_{m,m'} = k(x_m, x_{m'})$$

is positive semi-definite. As a consequence,

$$k(x_m, x_{m'}) \geq 0$$

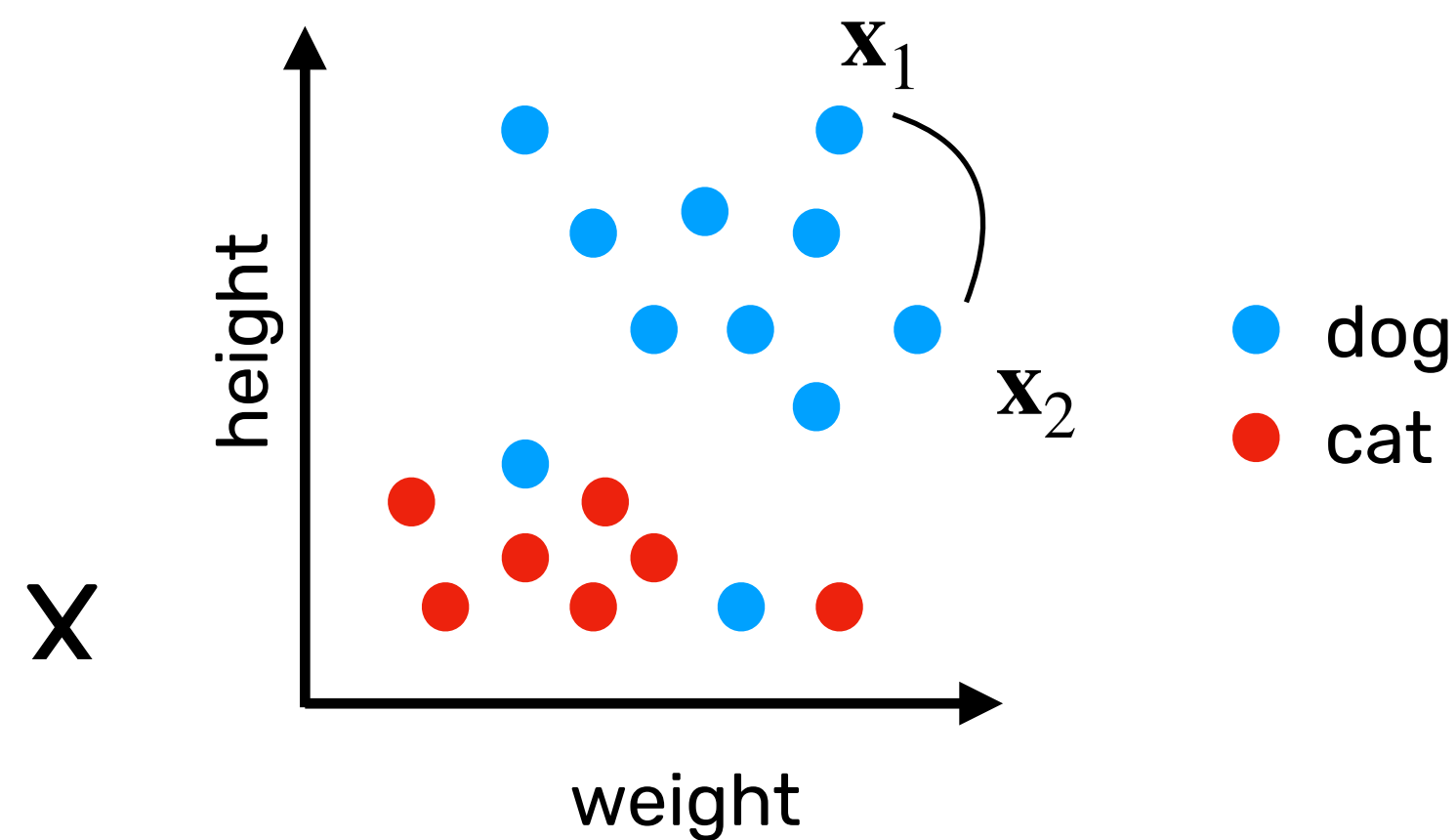
$$k(x_m, x_{m'}) = k(x_{m'}, x_m)$$

Classical Support Vector Machine

What is a kernel?

A kernel is a similarity measure defined over the input space.

Examples of kernels:



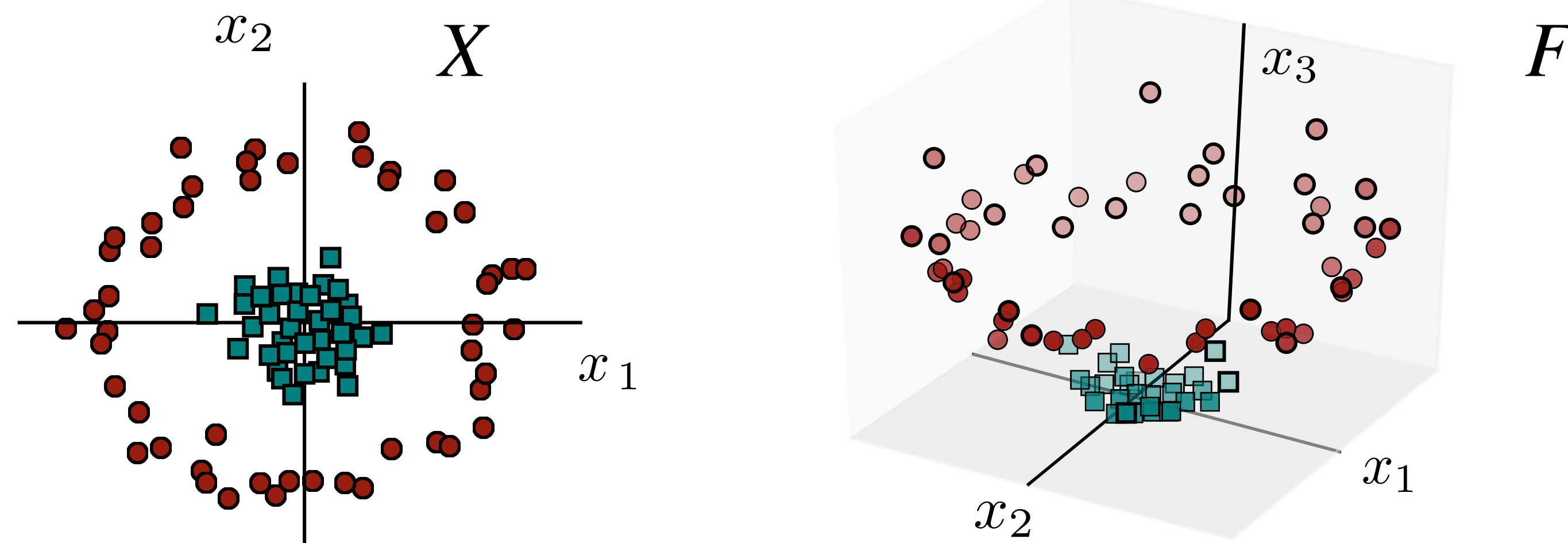
Name	Kernel	Hyperparameters
Linear	$\mathbf{x}^T \mathbf{x}'$	–
Polynomial	$(\mathbf{x}^T \mathbf{x}' + c)^p$	$p \in \mathbb{N}, c \in \mathbb{R}$
Gaussian	$e^{-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2}$	$\gamma \in \mathbb{R}^+$
Exponential	$e^{-\gamma \ \mathbf{x} - \mathbf{x}'\ }$	$\gamma \in \mathbb{R}^+$
Sigmoid	$\tanh(\mathbf{x}^T \mathbf{x}' + c)$	$c \in \mathbb{R}$

Classical Support Vector Machine

Kernels are closely related to feature maps.

For every valid kernel k , there exists a feature map $\phi : X \rightarrow F$ such that

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$



Classical Support Vector Machine

How are classical kernels evaluated?

By evaluating the kernel function

(NOT by mapping each data point to the corresponding feature space and then computing the inner product)

Name	Kernel	Hyperparameters
Linear	$\mathbf{x}^T \mathbf{x}'$	–
Polynomial	$(\mathbf{x}^T \mathbf{x}' + c)^p$	$p \in \mathbb{N}, c \in \mathbb{R}$
Gaussian	$e^{-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2}$	$\gamma \in \mathbb{R}^+$
Exponential	$e^{-\gamma \ \mathbf{x} - \mathbf{x}'\ }$	$\gamma \in \mathbb{R}^+$
Sigmoid	$\tanh(\mathbf{x}^T \mathbf{x}' + c)$	$c \in \mathbb{R}$

Classical Support Vector Machine

What is a support vector machine?

Support Vector Machines (SVMs) find/define a model:

$$f(\tilde{\mathbf{x}}) = \sum_{i=1}^M \alpha_i y_i k(\mathbf{x}_i, \tilde{\mathbf{x}}) + b$$

Support Vector Machines (SVMs) are commonly used for classification.

$\tilde{\mathbf{x}}$ is assigned class 1 if $f(\tilde{\mathbf{x}}) > 0$ and assigned class -1 if $f(\tilde{\mathbf{x}}) \leq 0$

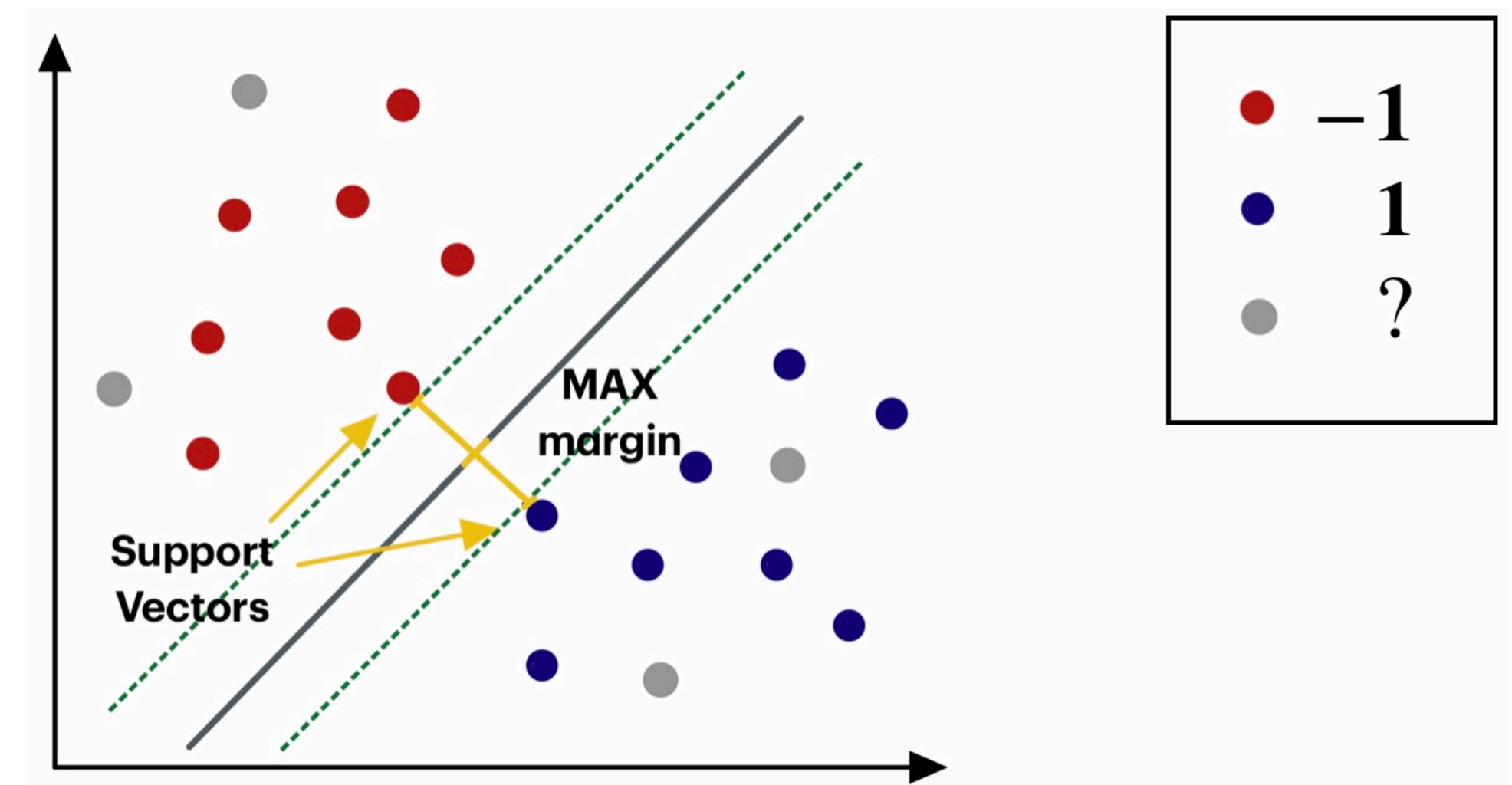
Classical Support Vector Machine

What is a support vector machine?

Support Vector Machines (SVMs) find/define a model:

$$f(\tilde{\mathbf{x}}) = \sum_{i=1}^M \alpha_i y_i k(\mathbf{x}_i, \tilde{\mathbf{x}}) + b$$

The parameters are optimised by solving a quadratic optimisation problem. The goal of the optimisation process is to find the decision boundary which maximises the margin



Quantum Support Vector Machine

What is quantum-enhanced support vector machine?

Quantum Computer

Classical Computer

Data

Quantum Kernel Estimation

1. Classical data is encoded into quantum states
2. The squared state overlap between the states is computed

Training of SVM

Decision boundary is constructed by solving a convex optimisation problem

Decision Rule



Quantum Kernel Estimation

If $U(\mathbf{x})$ defined some unitary operation that encodes the classical datum $\mathbf{x}$ into an n -qubit quantum state as

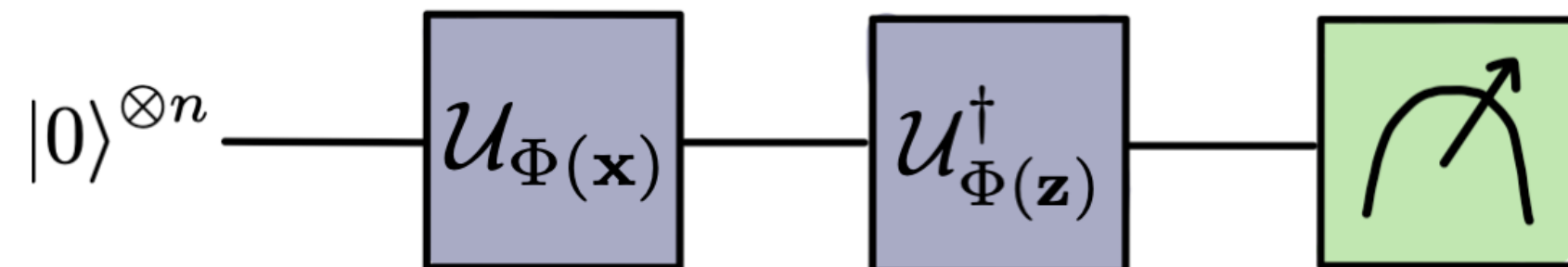
$$|\phi(\mathbf{x})\rangle = U(\mathbf{x}) |0\rangle^{\otimes n},$$

Then the kernel of two data points $\mathbf{x}$ and $\mathbf{z}$ is

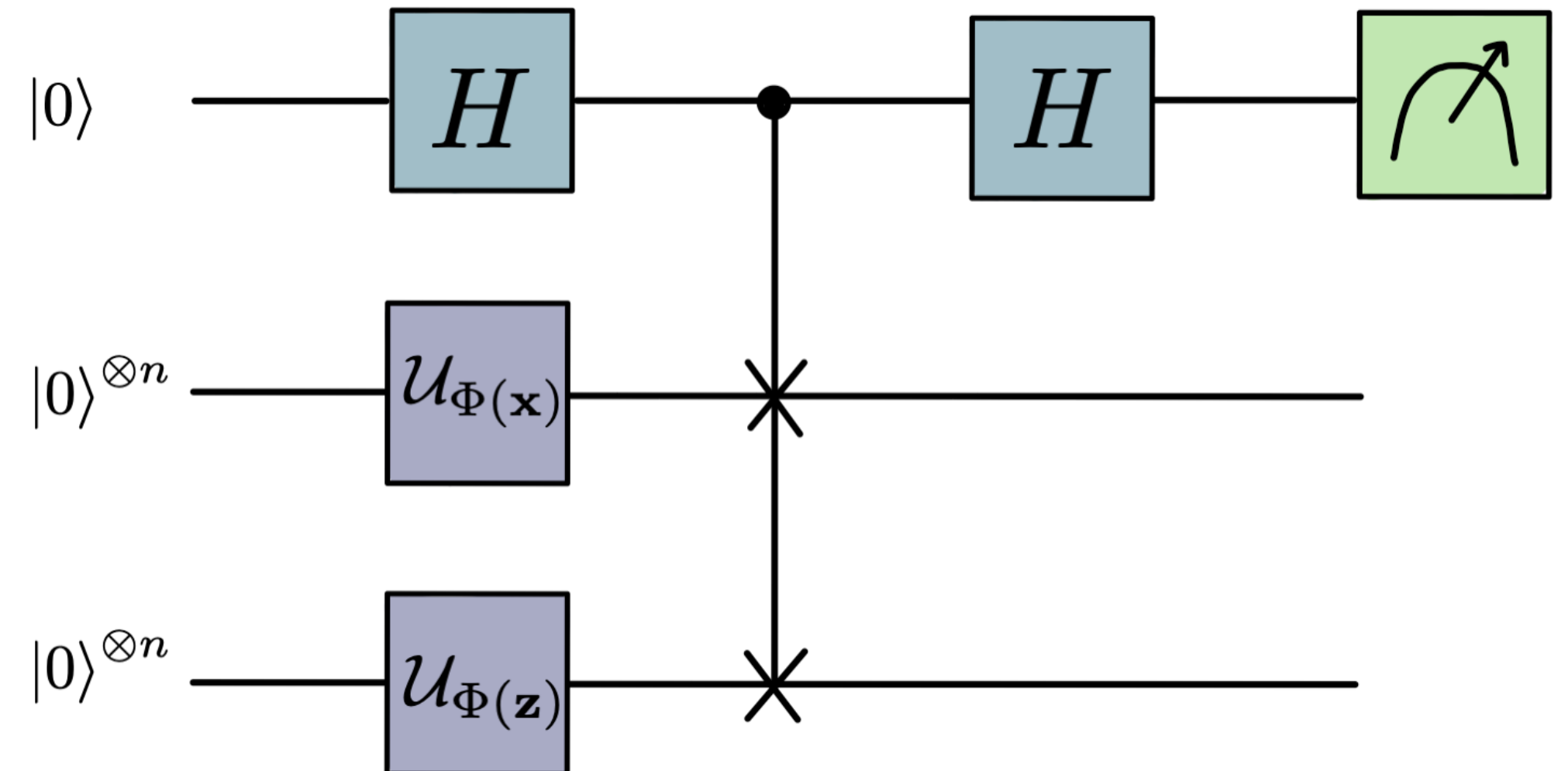
$$k(\mathbf{x}, \mathbf{z}) = |\langle \phi(\mathbf{z}) | \phi(\mathbf{x}) \rangle|^2$$

Quantum Kernel Estimation

Practically, this squared state overlap $k(\mathbf{x}, \mathbf{z}) = |\langle \phi(\mathbf{z}) | \phi(\mathbf{x}) \rangle|^2$ can be evaluated using the Inversion Test (most popular) or the SWAP-Test:



The Inversion Test



The SWAP-Test

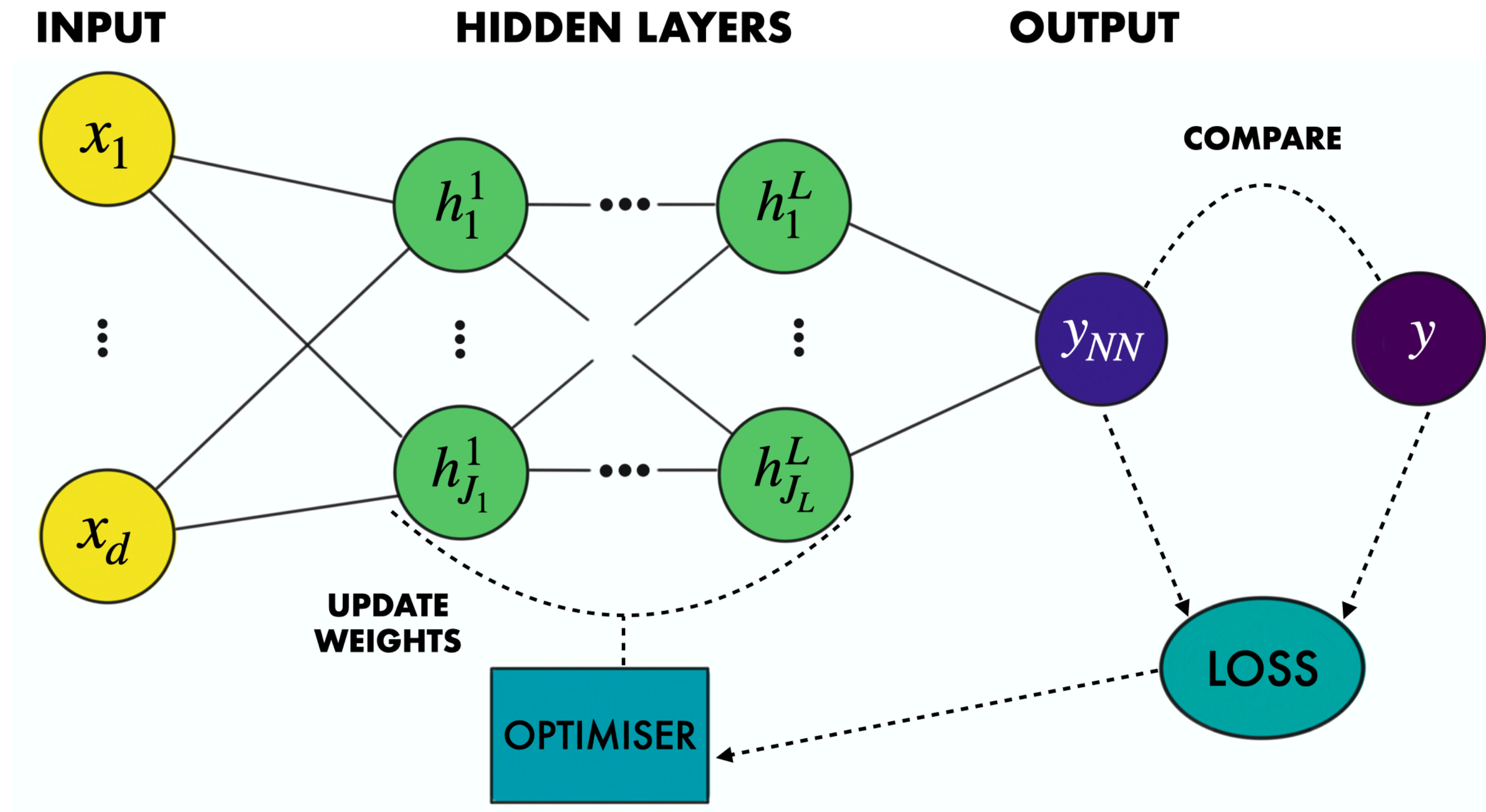
Quantum Kernel Estimation

Different kernels arise from different data encoding strategies.

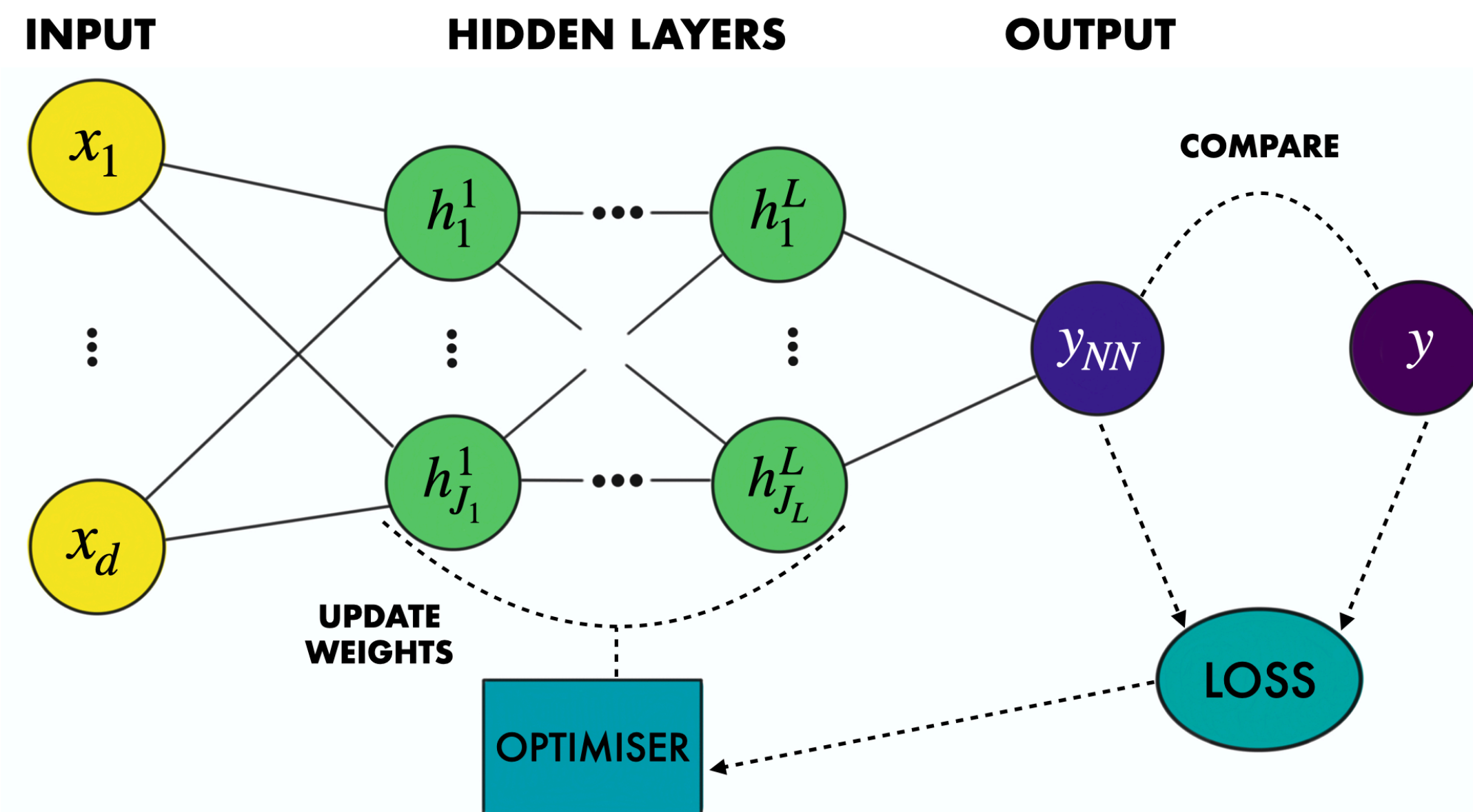
Some of the popular encoding strategies give rise to kernels that are analogous to some classical kernels.

Data Encoding Strategy	Feature Map	Kernel
Amplitude Encoding	$\mathbf{x} \rightarrow \sum_{i=1}^{2^n} x_i i\rangle$	$k(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x} \mathbf{z} \rangle ^2$
Rotation Encoding	$\mathbf{x} \rightarrow \sum_{q_1, \dots, q_n=0}^1 \prod_{k=1}^n \cos(x_k)^{q_k} \sin(x_k)^{1-q_k} q_1, \dots, q_n\rangle$	$k(\mathbf{x}, \mathbf{z}) = \prod_{k=1}^n \cos(x_k - z_k) ^2$
Coherent State Encoding	$\mathbf{x} \rightarrow \bigotimes_{i=1}^N \left(e^{-\frac{ x_i ^2}{2}} \sum_{k=0}^{\infty} \frac{x_i^k}{\sqrt{k!}} k\rangle \right)$	$k(\mathbf{x}, \mathbf{z}) = e^{- \mathbf{x}-\mathbf{z} ^2}$

Classical Neural Network



Classical Neural Network



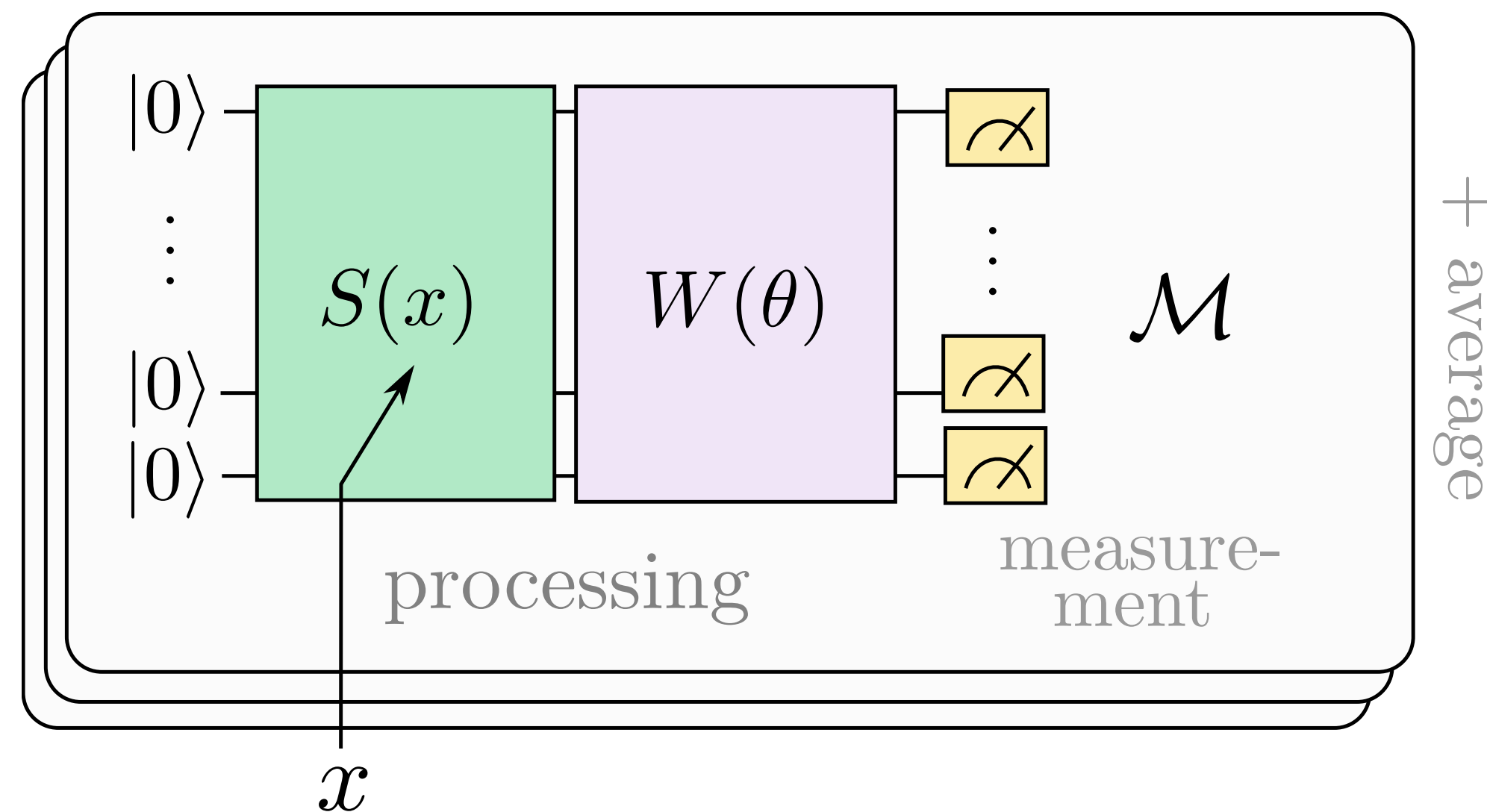
A fully-connected feed-forward neural network approximates some function $f = f(\mathbf{x})$ as

$$f_{NN}(\mathbf{x}) = \varphi_L(W_L \dots \varphi_2(W_2 \varphi_1(W_1 \mathbf{x})) \dots)$$

where each W_l is a weight matrix and each φ_l is a vectorised activation function. Typically, a neural network is trained by minimising a **loss function** which measures the discrepancy between the approximate function $f(\mathbf{x})$ and $f_{NN}(\mathbf{x})$.

Quantum Neural Network

A Parametrised Quantum Circuit (PQC) can be interpreted as a machine learning model.



The input $\mathbf{x}$ is encoded into a quantum state using some encoding block $S(\mathbf{x})$

$$\mathbf{x} \rightarrow S(\mathbf{x}) |0\rangle$$

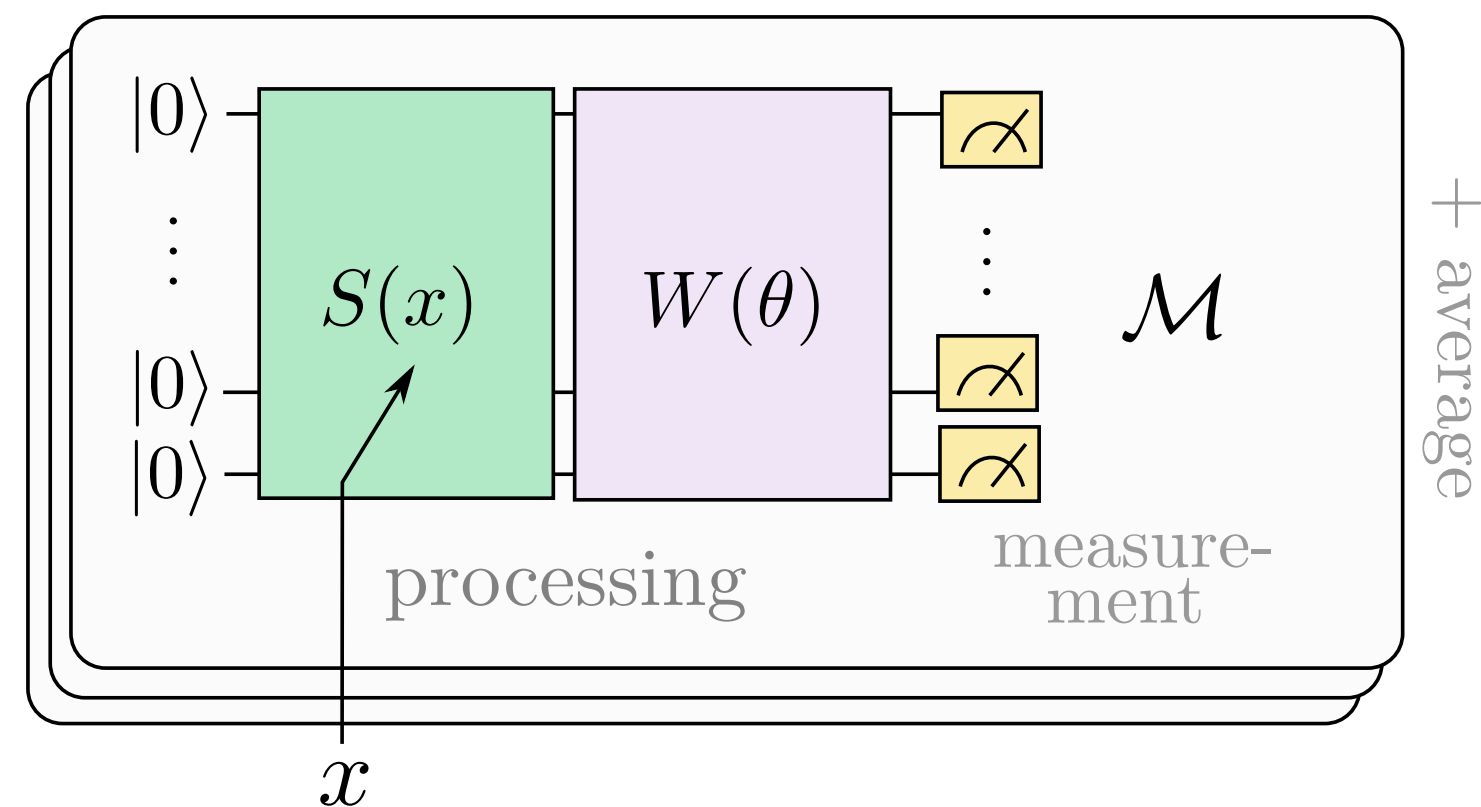
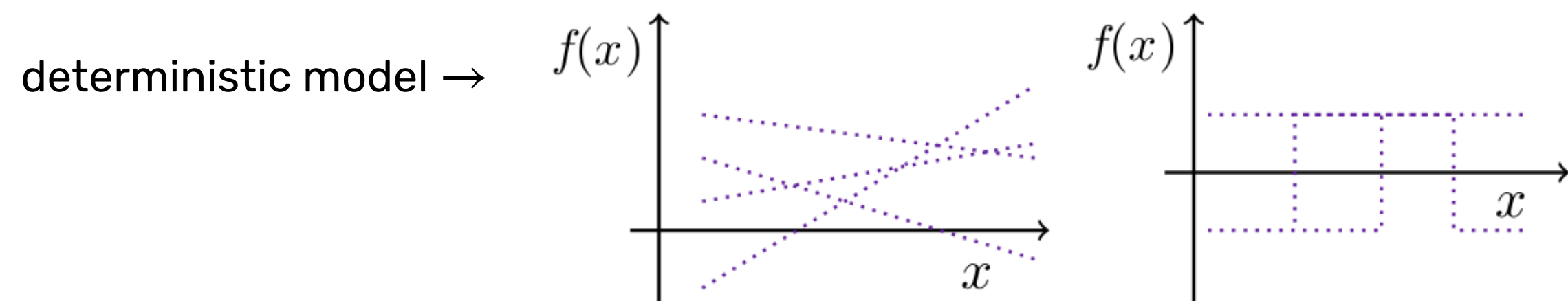
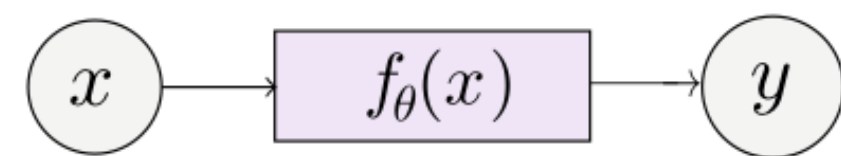
The encoded state is then processed using a parametrised block $W(\theta)$ with trainable parameters θ

$$\psi(\mathbf{x}, \theta) = W(\theta)S(\mathbf{x}) |0\rangle$$

The **expectation value** OR **outcome** of some measurement $\mathcal{M}$ is then interpreted as the output of the model

Quantum Neural Network

We can build a **deterministic model** by interpreting the expectation value of some measurement as the output of the model.



For example, if $M = \sum_i \mu_i |\mu_i\rangle\langle\mu_i|$ then we can interpret

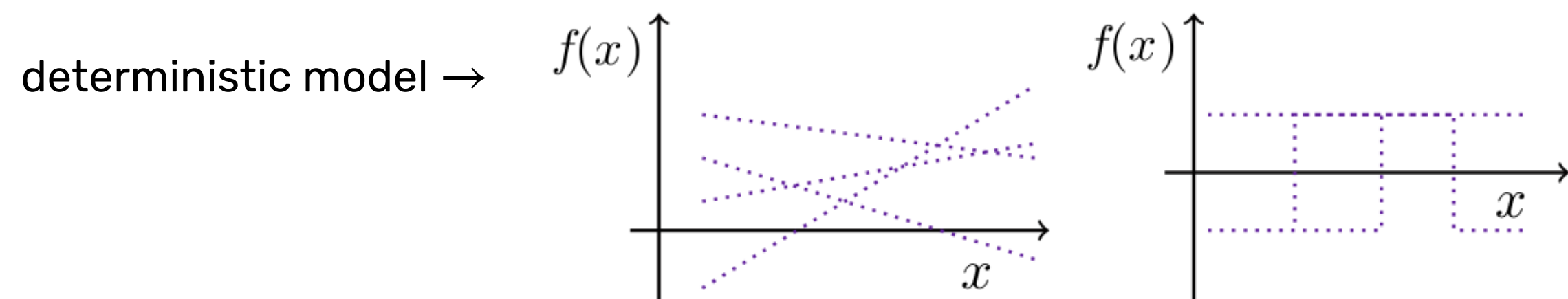
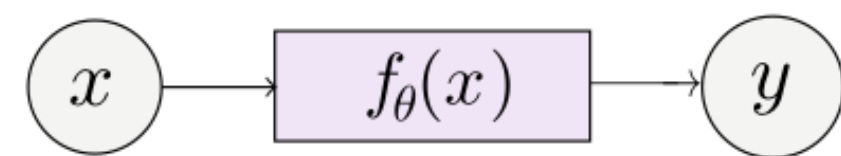
$$f_\theta(\mathbf{x}) = \langle \psi(\mathbf{x}, \theta) | \sum_i \mu_i |\mu_i\rangle\langle\mu_i| \psi(\mathbf{x}, \theta) \rangle$$

$$= \sum_i \mu_i p(\mu_i)$$

where $p(\mu_i) = |\langle \mu_i | \psi(\mathbf{x}, \theta) \rangle|^2$ is the probability of measuring outcome μ_i

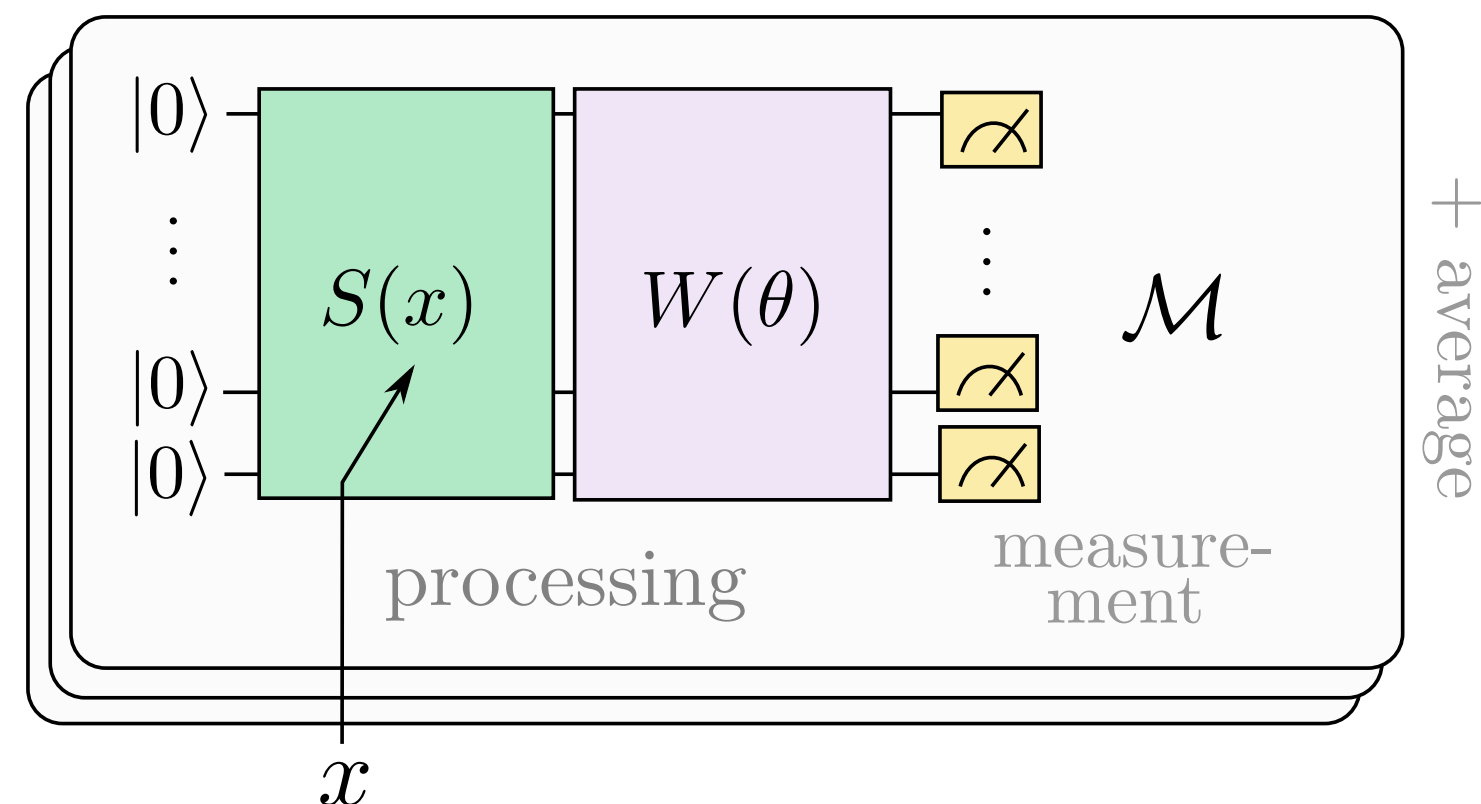
Quantum Neural Network

We can build a **deterministic model** by interpreting the **expectation value of some measurement as the output of the model.**



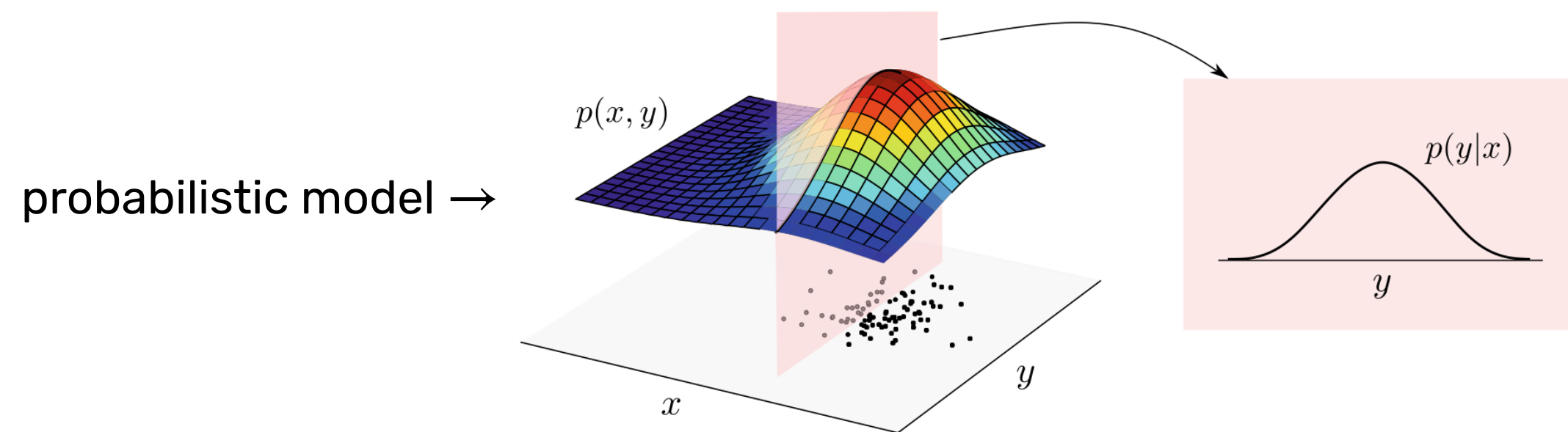
For example, if $M = \sigma_z = 1 |0\rangle\langle 0| - 1 |1\rangle\langle 1|$ then we can interpret

$$f_{\theta}(\mathbf{x}) = 1 |\langle 0 | \psi(\mathbf{x}, \theta) \rangle|^2 - 1 |\langle 1 | \psi(\mathbf{x}, \theta) \rangle|^2 \\ = 1p(1) - 1p(-1)$$



Quantum Neural Network

We can build a **probabilistic model** by interpreting the **outcome of some measurement as the output of the model**.

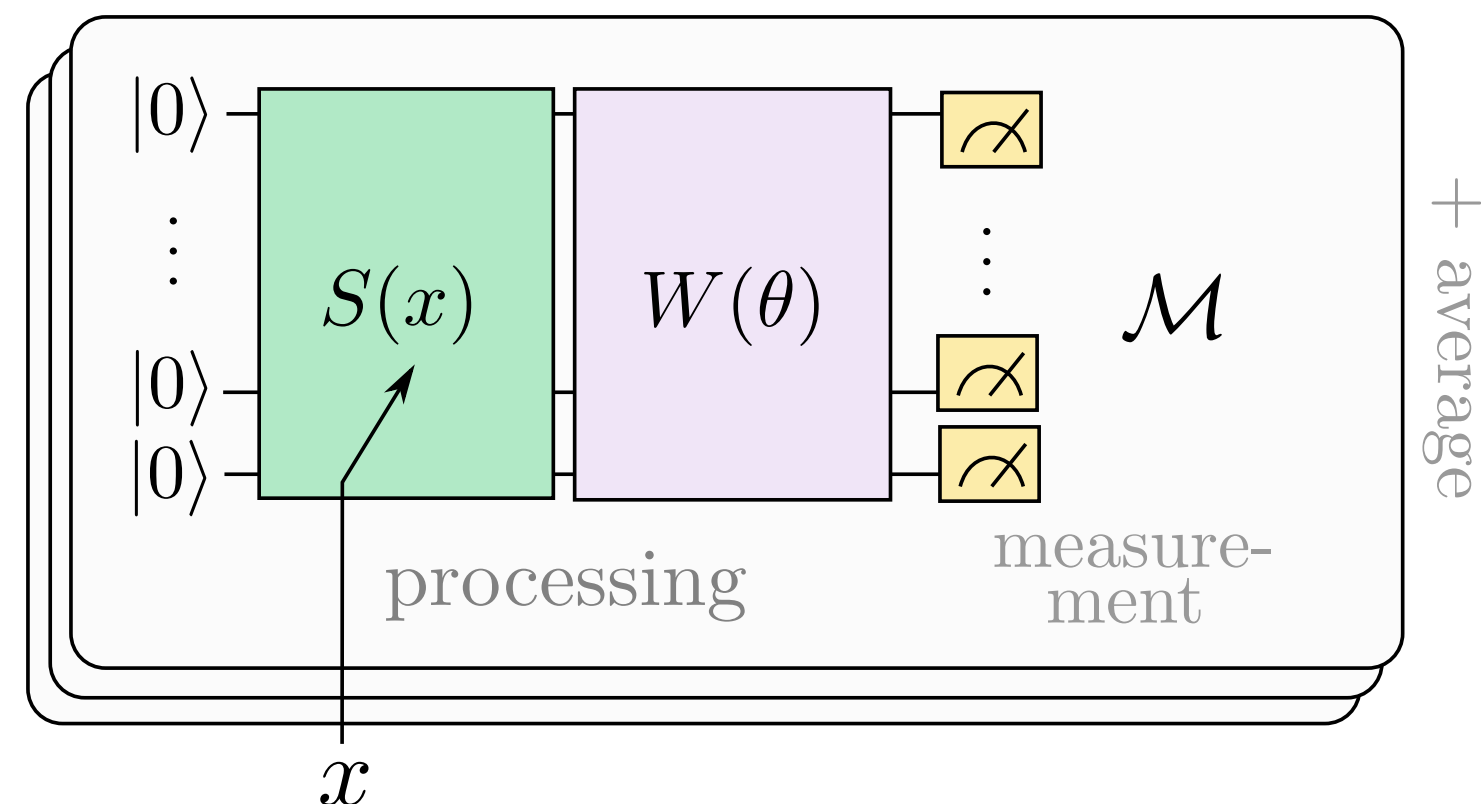


We associate each eigenvalue or outcome of measurement with a possible output y so that

$$y_i \longleftrightarrow \mu_i \quad \text{for} \quad M = \sum_i \mu_i |\mu_i\rangle \langle \mu_i|.$$

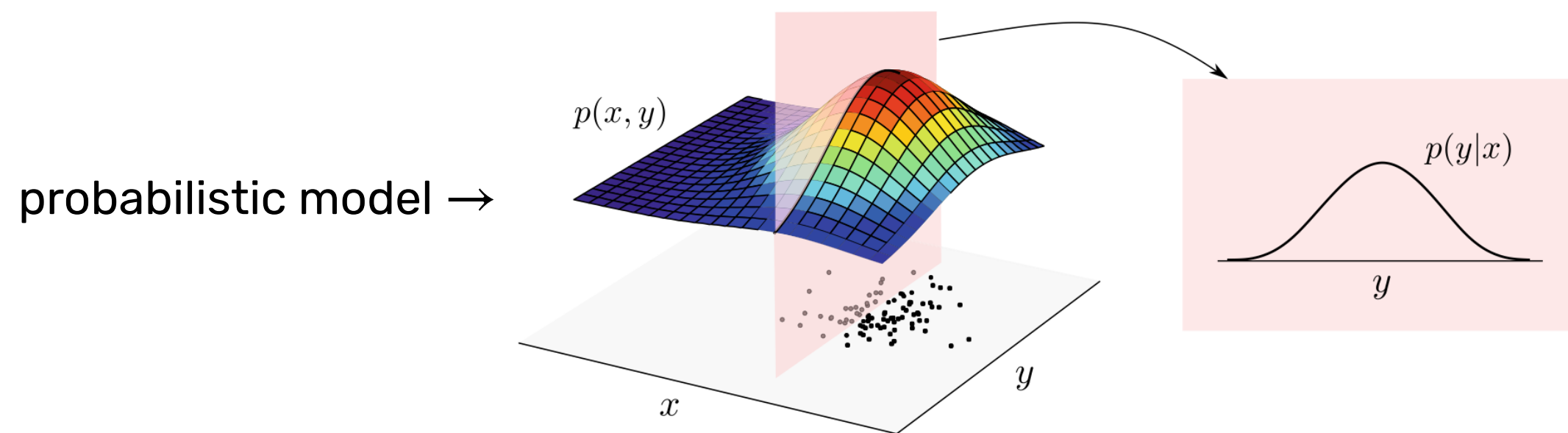
A probabilistic quantum model for a conditional distribution is defined by:

$$p_\theta(y | \mathbf{x}) = |\langle y | \psi(\mathbf{x}, \theta) \rangle|^2 = p(|y\rangle)$$

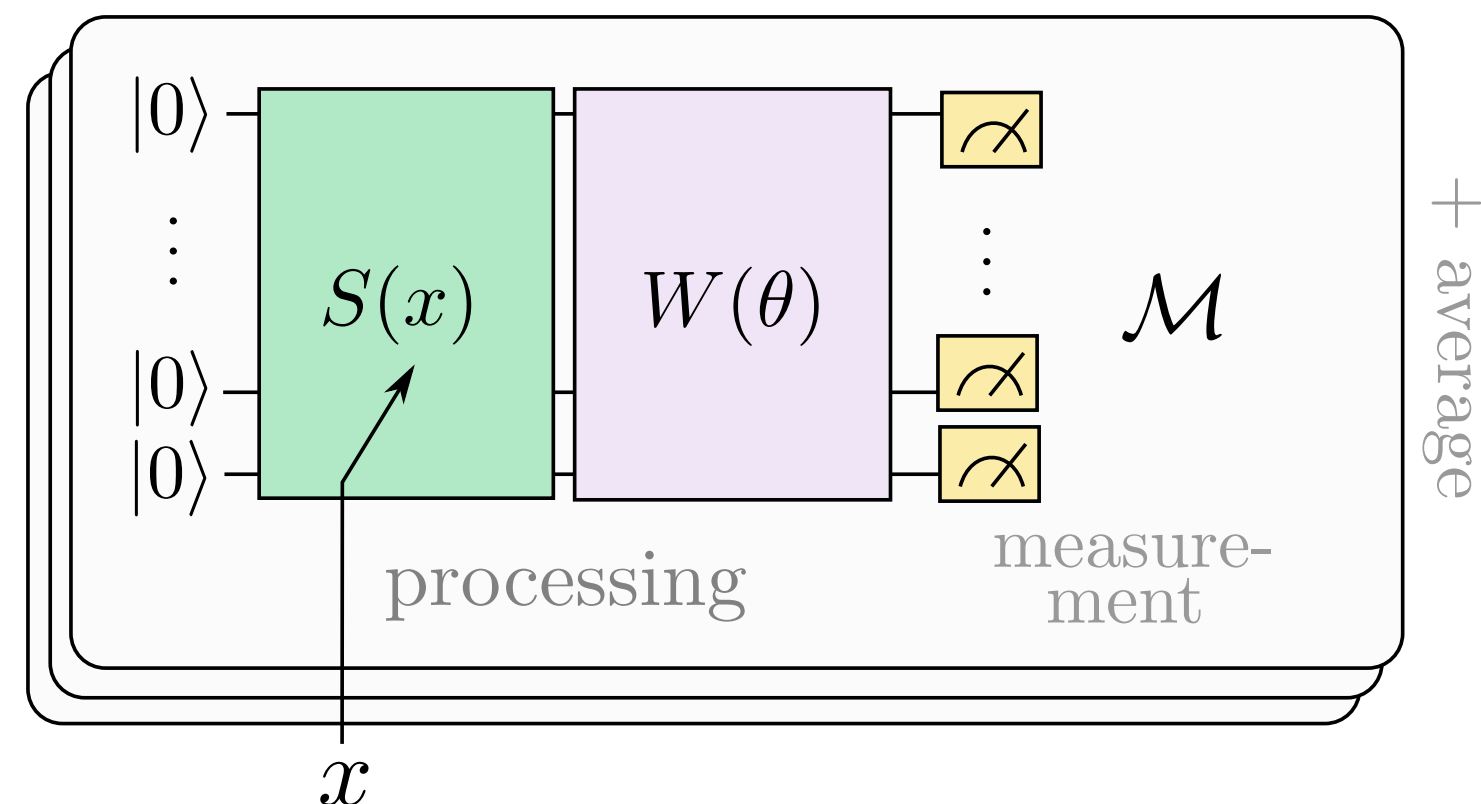


Quantum Neural Network

We can build a **probabilistic model** by interpreting the **outcome of some measurement as the output of the model**.



For example, if we have to perform binary classification and we encode and process our data on a single qubit, we can associate the labels $y = 1$ and $y = -1$ with the outcomes of a Pauli-Z measurement: $\sigma_z = 1 |0\rangle\langle 0| - 1 |1\rangle\langle 1|$

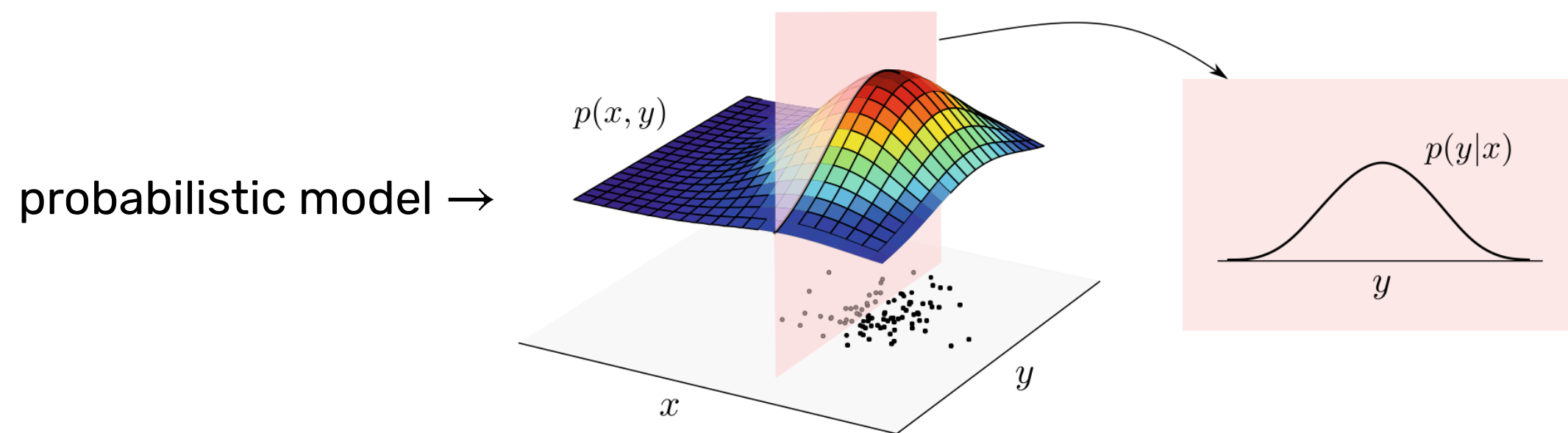


$$p_{\theta}(1 | \mathbf{x}) = |\langle 0 | \psi(\mathbf{x}, \theta) \rangle|^2$$

$$p_{\theta}(-1 | \mathbf{x}) = |\langle 1 | \psi(\mathbf{x}, \theta) \rangle|^2$$

Quantum Neural Network

We can build a **probabilistic model** by interpreting the **outcome of some measurement as the output of the model.**



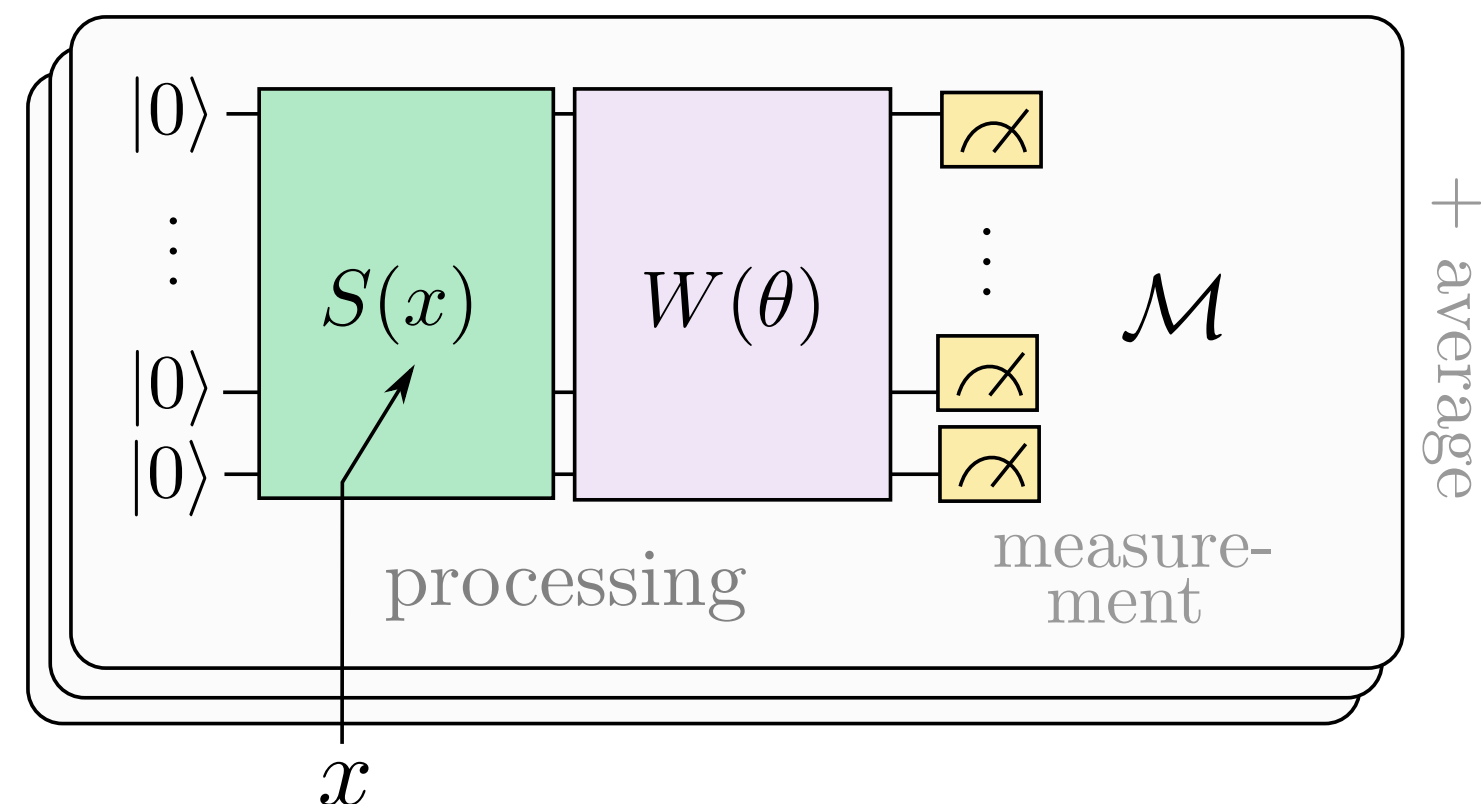
For example, if we have to perform binary classification and we encode and process our data in 2 qubits, we can use **parity**:

even parity:

$$p_{\theta}(1 | \mathbf{x}) = p(|00\rangle) + p(|11\rangle)$$

odd parity

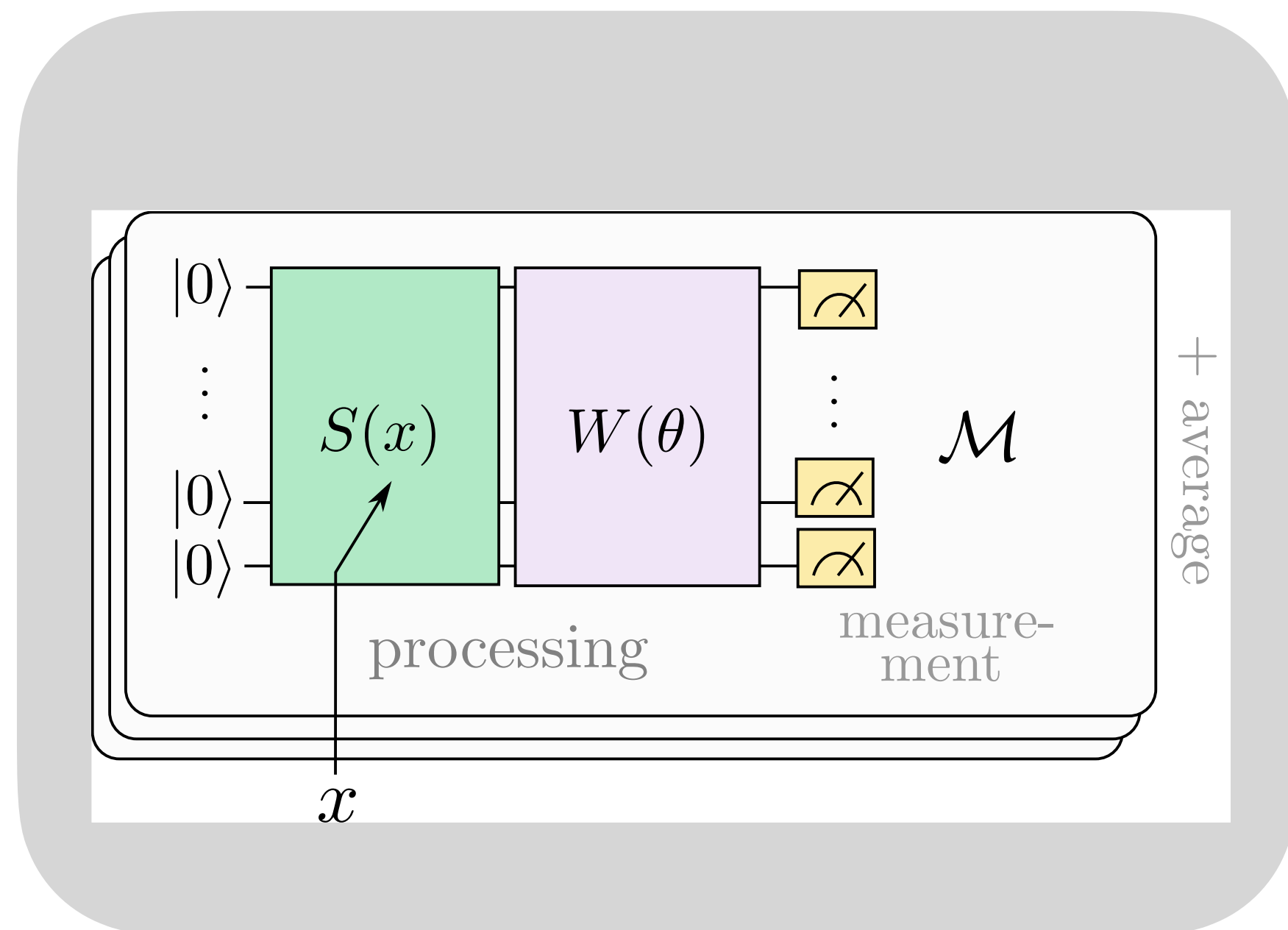
$$p_{\theta}(-1 | \mathbf{x}) = p(|01\rangle) + p(|10\rangle)$$



Quantum Neural Network

Training a quantum neural network

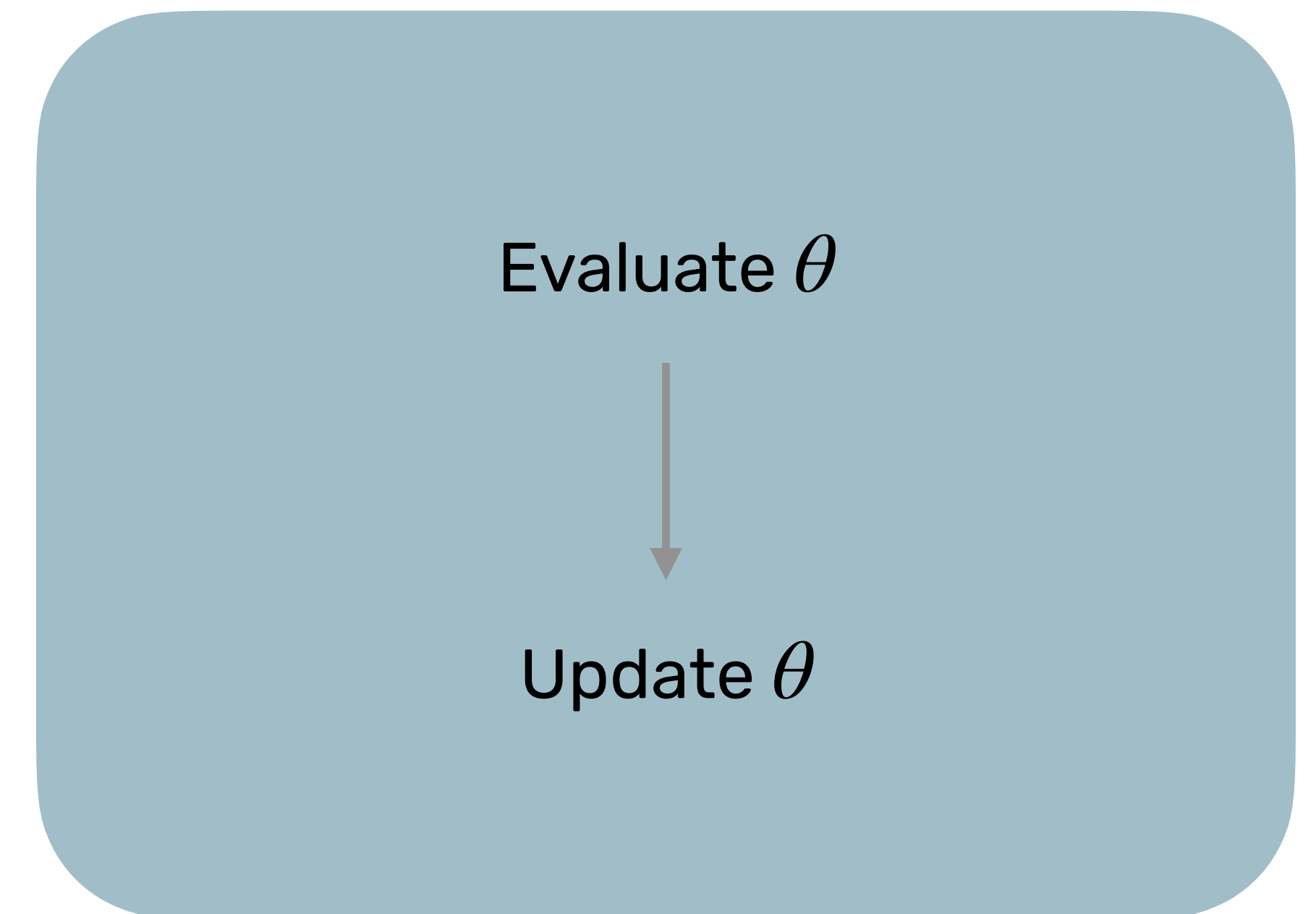
Quantum Computer



Output $f_{\theta}(\mathbf{x})$

Input New θ

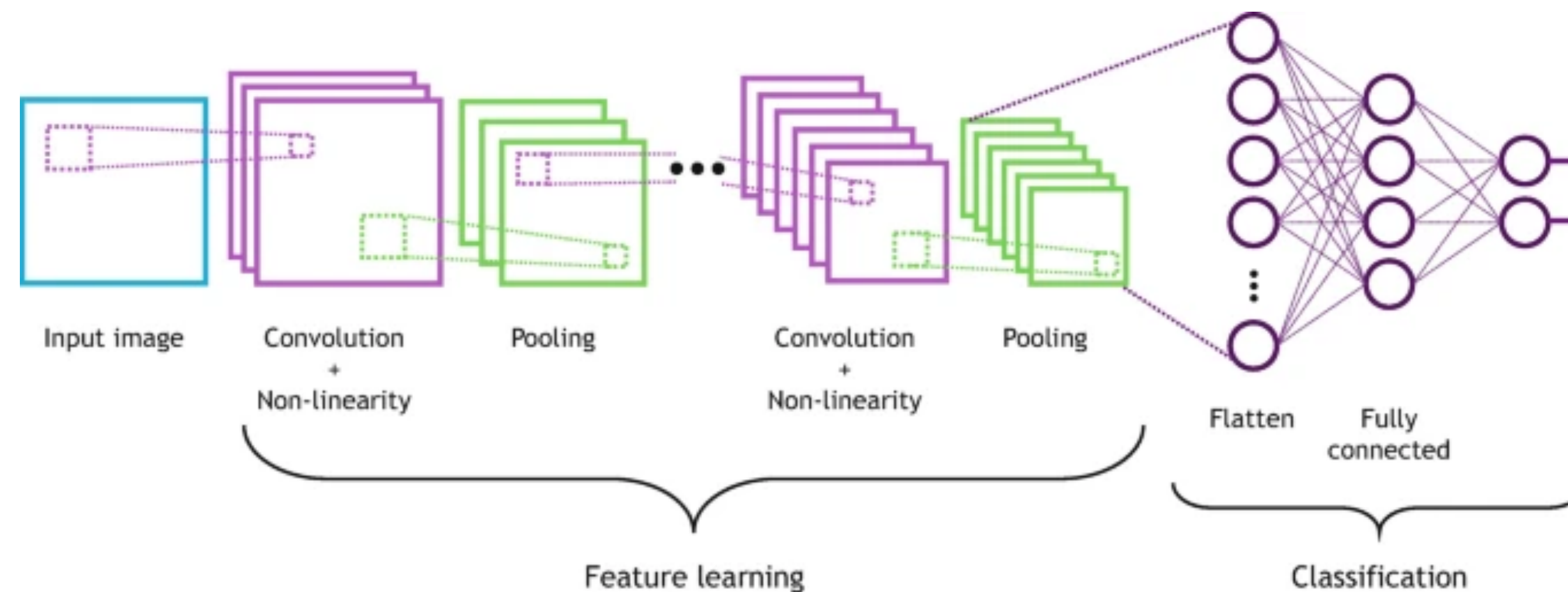
Classical Computer



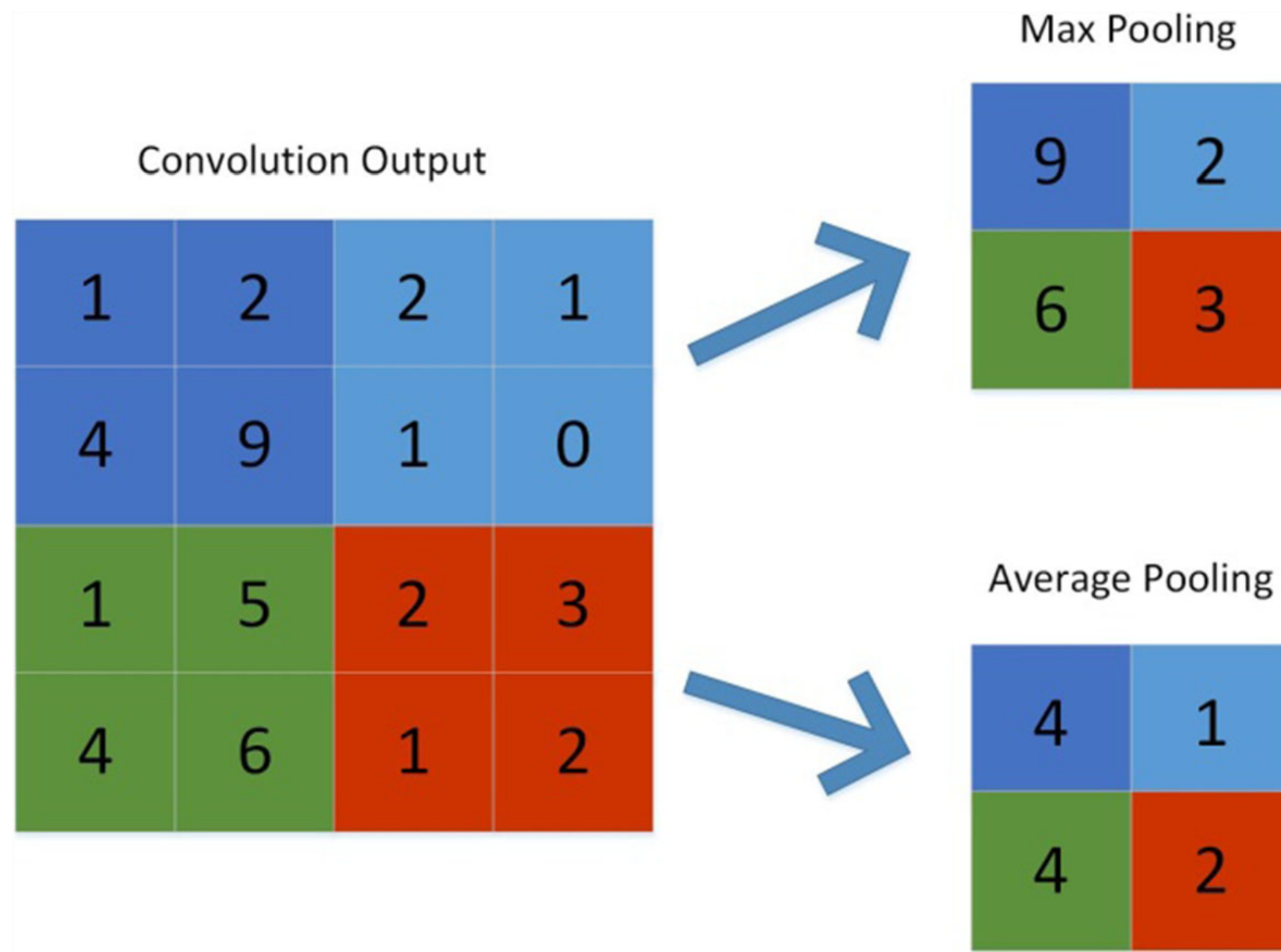
Classical Convolutional Neural Network

Convolutional Neural Networks (CNNs) are a type of classical machine learning model often used in computer vision and image processing applications.

The structure of CNNs consists of applying alternating **convolutional layers** (plus an activation function) and **pooling layers** to an input array, typically followed by some fully connected layers before the output.



Classical Convolutional Neural Network



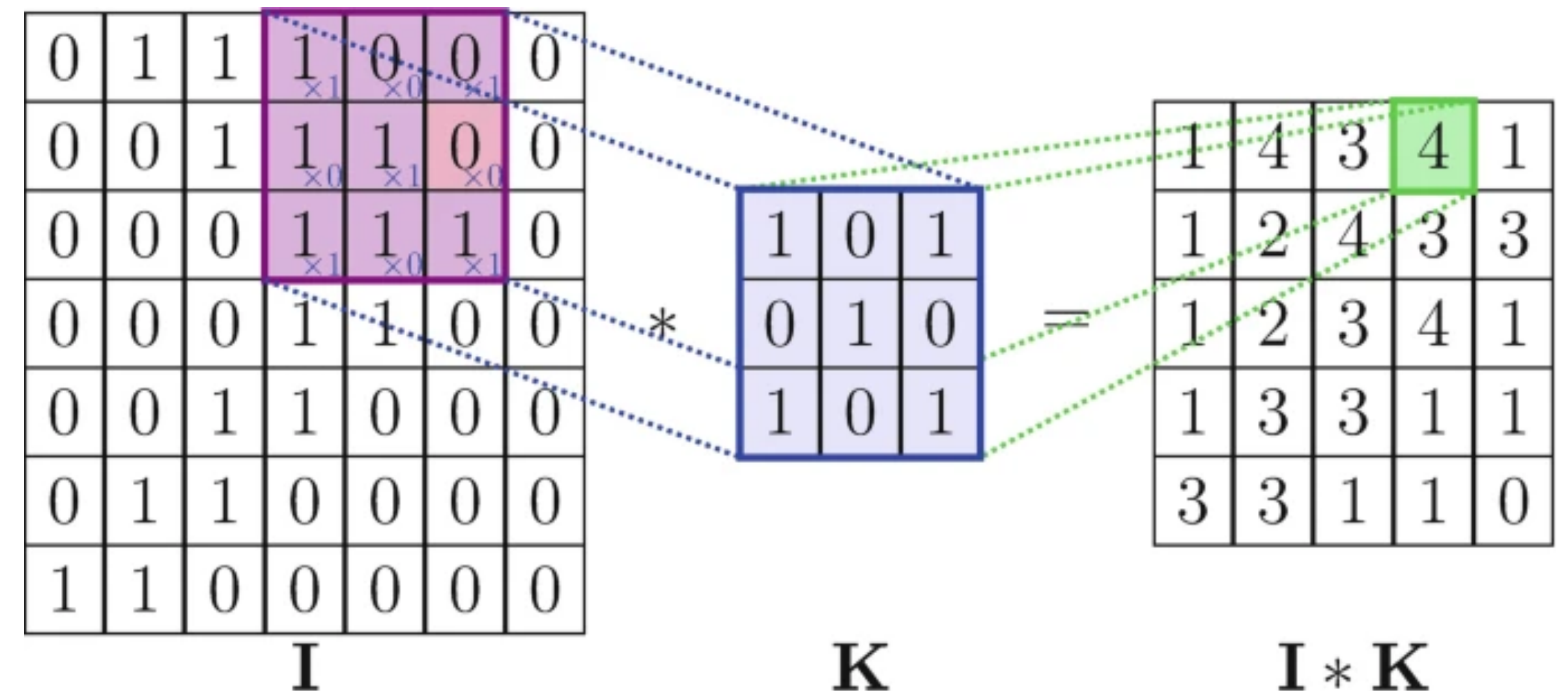
Pooling Layers: downsample the results of these convolutions to extract the most relevant features and reduce the size of the data, making it easier to process in subsequent layers.

Common pooling methods involve replacing blocks of the data with their maximum or average values.

Classical Convolutional Neural Network

Convolutional Layers: sweep across the input array and apply different filters (often 2x2 or 3x3 matrices) block by block.

These are used to detect specific features of the image wherever they might appear.

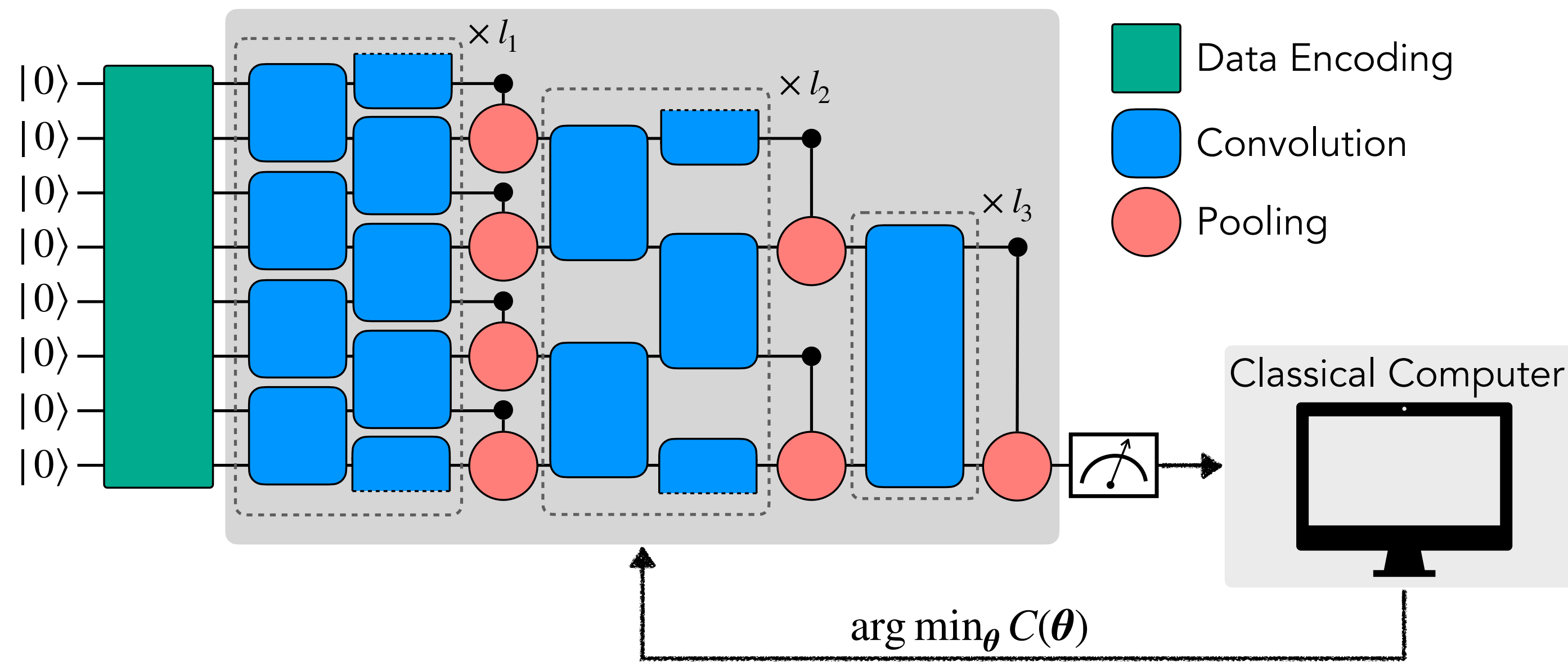


Quantum Convolutional Neural Network

Quantum Convolutional Neural Networks (QCNNs) are analogous to CNNs.

In QCNNs, classical data is first encoded onto a quantum computer.

The convolutional filters and pooling layers applied thereafter are **parametrised gates**.



The convolutional operations are just parametrised unitary rotations, applied to neighbouring qubits.

The pooling operations layers are controlled unitary transformations, activated when the corresponding control qubit is 1.

The analogue of a fully-connected layer in a CNN is a multi-qubit operation on the remaining qubits before the final measurement

Thank you
Any Questions?